# “SMART PARKING MANAGEMENT SYSTEM”

**A PROJECT REPORT**

Submitted in partial fulfilment of the

Requirements of the award of the Degree of

**BACHELOR OF SCIENCE (COMPUTER SCIENCE)**

**BY**

VEDANG TUKARAM KADAM

SEAT NUMBER – 2653021

**Under the esteemed guidance of**

**Mr. Yuvraj Wagh**

**DEPARTMENT OF COMPUTER SCIENCE**

**VPM’S R Z SHAH COLLEGE OF ARTS, SCIENCE & COMMERCE**

**(Affiliated to University of Mumbai)**

**NACC Accredited “A” Grade – First Cycle**

**MULUND – 400081**

**MAHARASHTRA**

**2026 - 2027**

**VPM'S R Z SHAH COLLEGE OF ARTS, SCIENCE & COMMERCE**

**(Affiliated to University of Mumbai)**

**NACC Accredited "A" Grade – First Cycle**

**MULUND – 400081**

**MAHARASHTRA**

**2026 - 2027**

**DEPARTMENT OF COMPUTER SCIENCE**

# CERTIFICATE

This is to certify that the project entitled. "Smart Parking Management System", is bonafide work of **VEDANG TUKARAM KADAM** bearing Seat No – **2653021** submitted is partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE** in **COMPUTER SCIENCE** from University of Mumbai.

Signature of Project Guide Coordinator

Signature of H.O.D

Signature of External Examiner

# DECLARATION

I hereby declare that the project entitled "SMART PARKING MANAGEMENT SYSTEM" submitted to **VPM'S R Z SHAH COLLEGE OF ARTS, SCIENCE & COMMERCE (Affiliated to University of Mumbai) Mulund,** has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirement for the award of the degree of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** to be submitted as 5th Semester project as part of my curriculum.

Name and Signature of the Student

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Prof. Yuvraj Wagh**, for his valuable guidance, encouragement and constructive suggestions throughout the development of this project. His support helped me understand the project requirements and complete the work in a systematic manner.

I am also thankful to the Head of the Department and all the faculty members of the **Department of Computer Science**, and to **VPM's R. Z. Shah College of Arts, Science and Commerce**, for providing the knowledge, resources and academic environment needed to complete this work.

This project gave me the opportunity to apply my theoretical knowledge to a practical problem and improved my understanding of web application development, map-based interfaces, online payment workflows and software project documentation.

Finally, I extend my thanks to my family and friends for their continuous support and motivation.

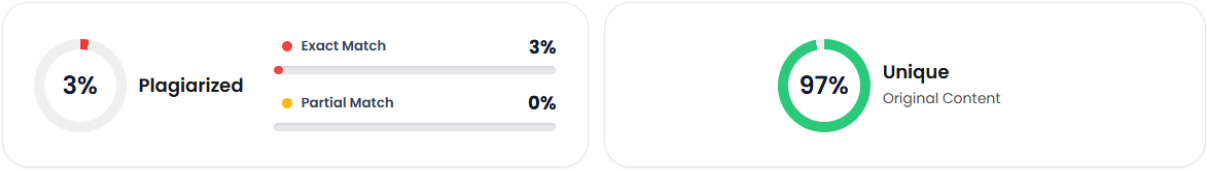
# ABSTRACT

Finding a parking space in a busy city is a daily difficulty for drivers. Without reliable information about where parking is available and what it costs, drivers spend time and fuel circling commercial areas, while parking operators have no convenient way to let customers plan and reserve a space in advance.

This project presents the design and implementation of **City Park – Smart Parking Management System**, a web application that lets a registered driver search for parking lots on an interactive map, compare them by price and distance, and see how many spots are recorded as available in each lot. For a selected lot the driver chooses a start time and a duration, the system calculates the parking fee automatically (for example, ₹40 per hour for two hours is shown as ₹80.00), and the booking is paid through Stripe checkout. The application also provides a My Bookings page, a profile and appearance (light/dark) settings page, a payment page, and a security page with biometric unlock, change-password, sign-out and delete-account options. Login is available through email and password or through a Google account.

Availability figures in City Park are recorded counts held by the system; the project does not use physical sensors or cameras. Payments in the current build run in Stripe test mode. The report covers the problem definition, requirements, system design with UML diagrams, implementation, results, testing, and the conclusion with limitations and future scope.

# PLAGIARISM REPORT



# TABLE OF CONTENTS


| S. No. | Section / Topic | Page No. |
|---|---|---|
| * | Certificate | |
| * | Declaration | |
| * | Acknowledgement | |
| * | Abstract | |
| * | Table of Contents | |
| * | List of Figures | |
| * | List of Tables | |
| 1. | INTRODUCTION | |
| 1.1 | Background | |
| 1.2 | Problem Statement | |
| 1.3 | Objectives | |
| 1.4 | Purpose | |
| 1.5 | Scope | |
| 2. | SURVEY OF TECHNOLOGIES | |
| 2.1 | Technology Stack Overview | |
| 2.2 | Why This Stack Was Chosen | |
| 2.2.1 | JavaScript | |
| 2.2.2 | React.js | |
| 2.2.3 | NoSQL Database | |
| 2.3 | Comparative Study | |
| 2.3.1 | GUI / Interface Framework Comparison | |
| 2.3.2 | Database Comparison | |

| | | |
|---|---|---|
| 2.3.3 | Programming Language Comparison | |
| 3. | REQUIREMENTS AND ANALYSIS | |
| 3.1 | Software Requirements | |
| 3.2 | Hardware Requirements | |
| 3.3 | Functional Requirements | |
| 3.4 | Non-Functional Requirements | |
| 3.5 | Project Scheduling | |
| 4. | DEVELOPMENT MODEL | |
| 4.1 | Waterfall Model | |
| 4.2 | User Registration and Login | |
| 4.3 | Parking Slot and Status Management | |
| 4.4 | Checking Parking Availability | |
| 4.5 | Booking / Slot Allocation | |
| 4.6 | Vehicle Entry, Exit and Payment | |
| 4.7 | Administration and Testing | |
| 5. | SYSTEM DESIGN | |
| 5.1 | Module Description | |
| 5.2 | Use Case Diagram | |
| 5.3 | Class Diagram | |
| 5.4 | Sequence Diagram | |
| 5.5 | Data Flow Diagram | |
| 5.5.1 | DFD Level 0 | |
| 5.5.2 | DFD Level 1 | |
| 5.5.3 | DFD Level 2 | |
| 5.6 | Entity–Relationship Diagram | |
| 5.7 | Data Dictionary | |
| 5.8 | Deployment Diagram | |

| | | |
|---|---|---|
| 5.9 | System Flowchart | |
| 6. | IMPLEMENTATION AND TESTING | |
| 6.1 | Key Code Snippets | |
| 6.1.1 | Database Connection and Table Creation | |
| 6.1.2 | User Registration | |
| 6.1.3 | Login Validation | |
| 6.1.4 | Checking Parking Availability | |
| 6.1.5 | Creating a Booking / Slot Allocation | |
| 6.1.6 | Updating Slot Status | |
| 6.1.7 | Vehicle Session / Payment Record | |
| 6.2 | User Interface Screenshots | |
| 6.3 | Testing Strategy | |
| 6.3.1 | Test Case Matrix | |
| 7. | CONCLUSION AND FUTURE SCOPE | |
| 7.1 | Conclusion | |
| 7.2 | Limitations | |
| 7.3 | Future Enhancements | |
| 8. | REFERENCES / BIBLIOGRAPHY | |
| 8.1 | References | |
| 9. | APPENDIX | |
| A | Application Startup Configuration | |
| B | Main Application Modules | |
| C | How to Run the Project | |

# LIST OF FIGURES


- Fig. 5.1 Use Case Diagram
- Fig. 5.2 Class Diagram
- Fig. 5.3 Sequence Diagram
- Fig. 5.4 DFD Level 0
- Fig. 5.5 DFD Level 1
- Fig. 5.6 DFD Level 2
- Fig. 5.7 Entity–Relationship Diagram
- Fig. 5.8 Deployment Diagram
- Fig. 5.9 System Flowchart
- Fig. 6.1–6.10 Application Screenshots and Test Output


# LIST OF TABLES


- Software Requirements
- Hardware Requirements
- Functional Requirements
- Project Scheduling
- Gantt Chart
- Data Dictionary
- Test Case Matrix

# 1. INTRODUCTION

City Park is a web-based smart parking management and reservation system. In large cities such as Mumbai and Navi Mumbai, parking space near commercial areas, malls and business districts is limited, and drivers often have no way of knowing in advance whether a lot has room, how much it charges, or how far away it is. This leads to wasted time, extra fuel consumption and congestion around popular locations.

City Park addresses this by bringing parking information into one place. The driver opens the application, signs in, and sees an interactive satellite map with priced markers for nearby parking lots, alongside a scrollable list showing each lot's name, address, price per hour, number of spots left and distance. The driver can search by area or lot name, choose a start date and time and a duration, and limit the results with a maximum-price-per-hour filter.

Selecting a lot opens a details panel with its address, a short description, the number of spots available out of the total (for example, 40 of 80 spots at Vashi Sector 17) and its amenities such as CCTV, attendant and EV charging. From the same panel the driver reserves a spot: the system shows the booking window and calculates the fee, and the driver completes the booking with "Pay and Reserve", which uses Stripe checkout. Bookings are collected in a My Bookings page, and the driver can manage a profile, appearance and security options from a dashboard.

The aim of City Park is to make finding and reserving parking quick, transparent and convenient, and to provide a base on which sensor-based live occupancy, operator dashboards and other services can be added later.

## 1.1 Background

Urban vehicle ownership keeps growing, but parking capacity grows far more slowly. Traditional parking lots depend on drivers arriving and finding out on the spot whether space exists. Paper registers and manual attendants make it hard to publish accurate availability, and drivers have little means to compare prices across locations.

Web and mobile technologies make it possible to present parking information digitally. Interactive maps can show where lots are located, simple filters can narrow them by price and time, and online payment services can confirm a reservation in advance. City Park applies these technologies to the parking problem for lots in and around Mumbai and Navi Mumbai, including locations such as Vashi Sector 17, Lower Parel (Phoenix), Worli Sea Face and Powai (Hiranandani).

The system keeps a record of each lot’s total and available spots and its hourly price. When a driver books, the booking is stored against the driver’s account so that it can be reviewed later. Because availability is a stored count and not a measurement from physical sensors, the figures are only as current as the data held by the system; this is stated as a limitation in Section 12.

## 1.2 Problem Statement

Drivers looking for parking commonly face the following problems: not knowing which lots have free spots, not knowing the price before arriving, having to travel from lot to lot, and having no way to reserve a spot in advance. Parking operators, in turn, lack a simple channel to present availability and accept advance bookings and payments.

The main problem is the lack of a single, easy-to-use platform that shows nearby parking lots on a map together with their price, distance and availability, lets the driver pick a time and duration, calculates the fee, and completes the reservation with an online payment. City Park addresses this problem by combining search, map, filtering, booking, payment and booking history in one application.

## 1.3 Objectives

The main objective of City Park is to simplify finding and reserving a parking space and to give drivers clear, advance information about price and availability.

The specific objectives are:

1. Secure Access: Allow a driver to create an account and log in using an email address and password or a Google account.

2. Parking Search: Allow the driver to search parking lots by area or lot name.

3. Map Display: Show nearby parking lots on an interactive map with priced markers and a “lots near you” indicator.

4. Filtering: Let the driver set the start date and time, the duration, and a maximum price per hour, and list only the lots that match.

5. Lot Details: Show each lot’s address, description, spots available out of total spots, price per hour, distance and amenities.

6. Booking: Let the driver reserve a spot by choosing a start time and a duration of 1, 2, 3, 4, 6 or 8 hours.

7. Fee Calculation: Calculate the parking fee automatically from the hourly price and the chosen duration and show the booking window.

8. Online Payment: Complete the reservation through Stripe checkout (test mode in the present build).

9. Booking History: Keep the driver’s active reservations and history in a My Bookings page.

10. Account Management: Provide profile, light/dark appearance, payment and security settings, including change password, sign out, delete account and an optional biometric unlock.

## 1.4 Purpose

The main purpose of the Smart Parking Management System is to provide an efficient, automated, and user-friendly solution for managing vehicle parking spaces. The system aims to reduce the time and effort required to find available parking slots, minimize traffic congestion, and improve the overall parking experience.

The Smart Parking Management System uses modern technology to monitor parking availability, manage vehicle entry and exit, and maintain accurate parking records.

They are specified as:

1. **Efficient Parking Management**: To manage parking spaces effectively and ensure maximum utilization of available parking slots.

2. **Reduce Time and Effort**: To help drivers quickly find available parking spaces without unnecessary searching.

3. **Real-Time Parking Availability**: To provide updated information about occupied and vacant parking slots.

4. **Automated Parking Operations**: To automate vehicle entry, exit, slot allocation, and parking record management.

5. **Reduce Traffic Congestion**: To minimize unnecessary vehicle movement caused by searching for parking spaces.

6. **Improve User Convenience**: To provide a simple and user-friendly interface for checking parking availability and managing parking activities.

7. **Enhance Security**: To maintain vehicle information and parking records for better monitoring and security.

8. **Reduce Manual Work**: To minimize human involvement in parking management and reduce errors associated with manual recordkeeping.

9. **Save Fuel and Reduce Pollution**: To reduce fuel consumption and environmental pollution by minimizing the time spent searching for parking.

10. **Digital Record Management**: To store and manage parking information systematically for easy access, tracking, and reporting.

## 1.5 Scope

The scope of City Park is to provide a centralized web application for searching, comparing and reserving parking lots.

The major areas covered are:

1. **User Accounts**: Registration and login through email/password or Google, with profile management.

2. **Search and Map**: Search by area or lot name, an interactive satellite map with priced markers and a locate-me control, and a synchronized list of lots.

3. **Filters**: Start date and time, duration and maximum price per hour.

4. **Lot Information**: Address, description, availability count, hourly price, distance and amenities (for example CCTV, attendant and EV charging).

5. **Reservation and Fee**: Selection of start time and duration, automatic fee calculation and display of the reservation window.

6. **Payment**: Stripe-based checkout; the Payment page explains that payments are processed securely through Stripe at checkout.

7. **Bookings**: A My Bookings page showing active reservations and history, with an empty state message when there are none.

8. **Settings and Security**: Profile (name and email), appearance toggle, change password, sign out, delete account and biometric (fingerprint/face) unlock option.

The system is intended for drivers and commuters in urban areas. Its limits are as follows:

Availability is a recorded count and not a live sensor reading. Payments in the current build use Stripe test mode, and real payments would need live Stripe keys and the system does not control physical barriers or entry gates. The report describes only features that appear in the working application.

# 2. SURVEY OF TECHNOLOGIES

The Survey of Technologies is an important part of the Smart Parking Management System project. It involves studying and analyzing various technologies, programming languages, frameworks, databases, and tools that can be used to develop an efficient and automated parking management system.

The Smart Parking Management System is designed to overcome the limitations of traditional parking methods by providing a digital platform for managing parking spaces, vehicle information, parking availability, and booking records. The system aims to reduce manual work, save time, improve accuracy, and provide a convenient parking experience for users.

In traditional parking systems, parking spaces are managed manually, which may lead to problems such as inefficient space utilization, difficulty in finding available parking slots, traffic congestion, and errors in maintaining vehicle records. Modern technologies help overcome these challenges by automating parking operations and providing accurate information about parking availability.

The development of a Smart Parking Management System can involve several technologies, including HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. Each technology plays an important role in improving the functionality, performance, and reliability of the system.

## 2.1 Technology Stack Overview

The Technology Stack Overview describes the various programming languages, frameworks, databases, and development tools used to build the Smart Parking Management System. A technology stack is a combination of software technologies that work together to develop, operate, and maintain an application.

The Smart Parking Management System is designed to automate parking operations, manage vehicle information, monitor parking slot availability, and provide an efficient parking experience. The system can be developed using frontend technologies, backend technologies, database management systems, and other supporting tools.

The selection of an appropriate technology stack is essential for ensuring system performance, security, reliability, scalability, and ease of maintenance.

## 2.2 Why This Stack Was Chosen

The selection of an appropriate technology stack is an important step in developing the Smart Parking Management System. The technologies are chosen based on factors such as ease of development, performance, reliability, security, cost-effectiveness, scalability, and user experience.

The proposed technology stack consists of HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. These technologies work together to create an efficient, secure, and user-friendly web-based parking management application.

The main reason for choosing this technology stack is to simplify the development process while ensuring that the system can manage parking slots, vehicle records, user information, and booking operations effectively.

### 2.2.1 JavaScript

JavaScript is a high-level, interpreted programming language widely used in web development to create dynamic, interactive, and user-friendly web applications. It is one of the core technologies used in modern website development, along with HTML and CSS.

HTML is responsible for defining the structure of a web page, CSS is used to design and style the page, and JavaScript adds functionality and interactivity to the application. It enables websites to respond to user actions, validate information, update content dynamically, and communicate with backend services.

In the Smart Parking Management System, JavaScript is used to support interactive website functionality. It helps users interact with the parking system and allows administrators to manage parking-related operations through the application interface.

JavaScript makes the application more responsive and convenient by reducing the need to reload the entire webpage whenever a user performs an operation.

### 2.2.2 React.js

React.js is an open-source JavaScript library used for developing interactive and dynamic user interfaces, especially for web applications. It was developed by Facebook (now Meta) and is widely used for building modern single-page applications (SPAs).

React.js allows developers to create reusable UI components. Instead of developing an entire webpage as one large block of code, the application can be divided into smaller and independent components. Each component is responsible for a particular part of the user interface.

React.js uses JavaScript and JSX (JavaScript XML) to define user interface elements. JSX allows developers to write HTML-like syntax within JavaScript code, making the development process easier and more organized.

In the Smart Parking Management System, React.js can be used to develop the frontend interface and manage the different screens and interactive components of the application. It helps create separate interfaces for users and administrators, display parking slot information, manage booking interactions, and present dashboard information.

### 2.2.3 NoSQL Database

NoSQL stands for "Not Only SQL." It refers to a category of database management systems designed to store, manage, and retrieve data without relying exclusively on the traditional relational database structure.

Unlike relational databases, which store information in tables consisting of rows and columns, NoSQL databases can store data in different formats, such as documents, key-value pairs, graphs, and wide-column structures.

NoSQL databases are commonly used in modern web applications because they provide flexibility, scalability, and efficient data management. They are suitable for applications that handle different types of information and require frequent data updates.

In the Smart Parking Management System, a NoSQL database can be used to store and manage information related to users, parking slots, reservations, payments, and administrative activities.

For example, a document-oriented NoSQL database can store parking slot information as individual documents containing details such as slot ID, parking area, availability status, and booking information.

The NoSQL database helps the application organize and retrieve parking-related information efficiently.

## 2.3 Comparative Study

A comparative study is an important part of software development because it helps identify and evaluate different technologies that can be used to develop an application. It involves comparing technologies based on their features, advantages, disadvantages, performance, flexibility, scalability, and suitability for the project.

The Smart Parking Management System is a web-based application designed to simplify parking management by providing facilities such as user registration, login, parking slot availability, slot booking, payment management, and administrative control.

Different technologies are available for developing web applications. Selecting suitable technologies is important to ensure that the application is user-friendly, reliable, efficient, and easy to maintain.

### 2.3.1 GUI / Interface Framework Comparison

A GUI (Graphical User Interface) is an interface that allows users to interact with a software application through visual elements such as buttons, menus, forms, icons, navigation bars, and dashboards.

In a Smart Parking Management System, the GUI plays an important role because it provides a platform through which users and administrators can perform different parking-related operations.

The user interface should be simple, attractive, responsive, and easy to understand. It should allow users to register, log in, check parking slot availability, book parking spaces, and make payments. Similarly, administrators should be able to manage parking slots, monitor bookings, and view payment information.

Different GUI and interface frameworks are available for developing web applications. Selecting an appropriate framework helps improve the application's usability, maintainability, and performance.

### 2.3.2 Database Comparison

A database is an essential component of a software application. It is used to store, organize, manage, and retrieve information whenever required. In a Smart Parking Management System, the database maintains important information related to registered users, parking slots, reservations, payment transactions, and administrative activities.

The database helps maintain accurate records and allows users and administrators to access the required information efficiently.

Different database management systems are available for developing web-based applications. Each database has its own features, advantages, limitations, and storage mechanisms. Selecting an appropriate database is important for ensuring data consistency, security, reliability, and scalability.

### 2.3.3 Programming Language Comparison

A programming language is used to develop the functionality, logic, and operations of a software application. It provides instructions that allow a computer system to perform specific tasks.

In a web-based application, programming languages are used to develop the frontend, backend, database operations, and communication between different application components.

The Smart Parking Management System is designed to simplify parking operations by providing features such as user registration, login, parking slot availability, slot booking, payment management, and an administrator dashboard.

Selecting an appropriate programming language is important because it affects application development, performance, security, scalability, and maintenance.

# 3. REQUIREMENTS AND ANALYSIS

Requirement analysis is an important phase in the development of the Smart Parking Management System. It involves identifying, understanding, and documenting the functional and non-functional requirements necessary for designing and implementing an efficient parking management application.

The Smart Parking Management System is designed to automate traditional parking operations such as vehicle registration, parking slot allocation, vehicle entry and exit management, parking fee calculation, and maintenance of parking records. The system aims to reduce manual work, improve accuracy, save time, and provide an organized method of managing parking spaces.

The proposed system uses JavaScript as the programming language, React.js for the graphical user interface, and NoSQL for database management. These technologies provide a suitable foundation for developing a desktop-based parking management application.

Requirement analysis ensures that the system meets the needs of users and administrators while maintaining reliability, security, efficiency, and ease of use.

## 3.1 Software Requirements

Software requirements specify the software, tools, platforms, and technologies needed to develop, operate, test, and maintain the Smart Parking Management System. These requirements ensure that the application functions properly and provides a smooth experience for both administrators and users.

City Park is a web-based Smart Parking Management System that allows users to view parking slot availability, book parking spaces, and make payments. Administrators can manage parking slots, monitor bookings, and oversee parking operations.

| Software | Purpose |
|---|---|
| JavaScript | Application logic and validation |
| NoSQL Database | Structured persistent storage |
| Browser / GUI runtime | Run the actual interface |
| Editor and Git | Development and version control |
| Dependencies | Exact packages from the project dependency file |

## 3.2 Hardware Requirements

The source report mentions an Intel Core i3 or equivalent processor, at least 4 GB RAM and 500 GB storage. These are reference specifications from that report, not independently verified minimums. Sensors, cameras and display boards are optional and required only for physical integrations.

| Hardware | Details |
| --- | --- |
| Computer | Source report mentions Intel Core i3 equivalent and 4 GB RAM |
| Storage | Source report mentions 500 GB; verify actual needs |
| Sensors / cameras / display | Optional, for connected hardware integrations only |

## 3.3 Functional Requirements

FR-01: Display or manage parking-slot records.

FR-02: Show each slot's recorded status.

FR-03: Validate user input.

FR-04: Read and write parking records through the configured database.

FR-05: If booking exists, allow allocation only when the slot meets availability rules.

FR-06: If an administrator module exists, restrict updates to authorized users.

FR-07: Payment and vehicle records are required only if those functions are implemented.

FR-08: Provide understandable success and error messages.

| ID | Requirement |
|---|---|
| FR-01 | Display/manage parking-slot records |
| FR-02 | Display recorded slot status |
| FR-03 | Validate inputs |
| FR-04 | Read/write database records |
| FR-05 | Booking/admin/payment only if implemented |

## 3.4 Non-Functional Requirements

Non-functional requirements are an essential part of the Smart Parking Management System because they define the quality, performance, security, reliability, and usability standards that the system should maintain. Unlike functional requirements, which describe what the system does, non-functional requirements specify how efficiently and effectively the system should perform its operations.

The Smart Parking Management System is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS to provide an efficient, reliable, and user-friendly desktop-based parking management application. The system manages vehicle registration, parking slot allocation, vehicle entry and exit records, parking fee calculations, and database operations.

Non-functional requirements ensure that the application performs smoothly, protects important information, minimizes errors, and provides a convenient experience for users and administrators.

## 3.5 Project Scheduling

The proposed project schedule covers eight weeks, including planning, development, testing and documentation.

| Week | Activity | Expected Outcome |
|---|---|---|
| 1 | Requirement gathering and study of existing parking apps | Finalized requirements |
| 2 | System design and architecture | UML diagrams and data model |
| 3 | User interface development | Login, dashboard and layout |
| 4 | Map, search and filters | Map with priced markers and lot list |
| 5 | Lot details, booking and fee calculation | Reservation panel with fee |
| 6 | Payment integration and My Bookings | Stripe checkout and booking history |
| 7 | Account settings, security and testing | Integrated, tested system |
| 8 | Documentation and final review | Completed project report |

## Gantt Chart

A Gantt chart represents project activities against time. It helps plan, schedule and monitor each phase of development. Shaded cells show the weeks in which an activity is carried out.

| Sr | Project Activity | W1 | W2 | W3 | W4 | W5 | W6 | W7 | W8 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | Requirement Analysis and Study | | | | | | | | |
| 2 | System Design | | | | | | | | |
| 3 | UI and Interface Development | | | | | | | | |
| 4 | Map, Search and Filter Module | | | | | | | | |
| 5 | Booking and Fee Calculation | | | | | | | | |
| 6 | Payment and My Bookings | | | | | | | | |
| 7 | Account Settings and Security | | | | | | | | |
| 8 | Testing and Debugging | | | | | | | | |
| 9 | Documentation and Final Submission | | | | | | | | |

# 4. DEVELOPMENT MODEL

A software development model is a systematic approach used to plan, design, develop, test, and maintain a software application. It defines the different phases of software development and provides guidelines for completing the project efficiently.

The Smart Parking Management System is a desktop-based application designed to automate parking operations such as vehicle registration, parking slot allocation, vehicle entry and exit management, parking fee calculation, and parking record maintenance.

The proposed system uses JavaScript as the programming language, React.js for graphical user interface development, and NoSQL for database management.

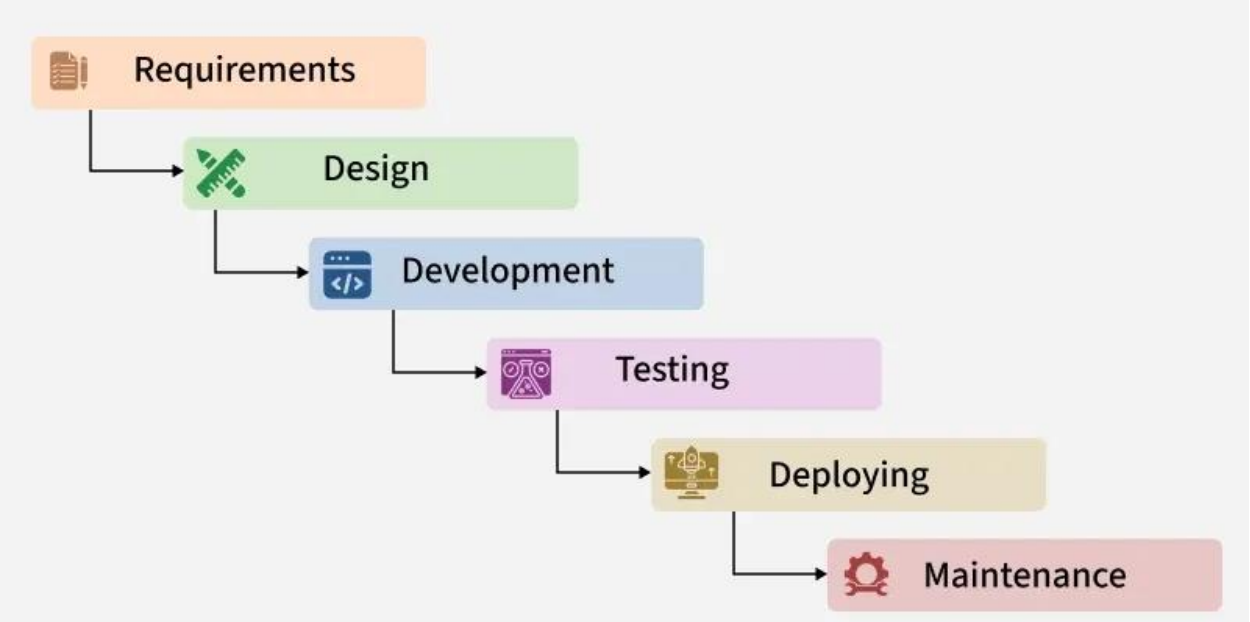
For this project, the Waterfall Model is selected as the proposed software development model because it provides a simple, structured, and sequential approach suitable for a project with clearly defined requirements.

## 4.1 Waterfall Model

The Waterfall Model is a traditional Software Development Life Cycle (SDLC) model in which development activities are divided into separate phases.

Each phase produces specific deliverables that are reviewed before proceeding to the next phase.

The Waterfall Model is suitable for the Smart Parking Management System when the core requirements, such as vehicle registration, parking slot management, fee calculation, and database operations, are defined before development begins.



## 4.2 User Registration and Login

User Registration and Login is an important module of the Smart Parking Management System. It provides a secure and organized mechanism for users and administrators to access the application. The main purpose of this module is to register new users, verify their identities, maintain user information, and prevent unauthorized access to parking management operations.

The proposed Smart Parking Management System is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. React.js provides the graphical interface for registration and login, JavaScript handles validation and authentication, and NoSQL stores user account information.

## 4.3 Parking Slot and Status Management

Parking Slot and Status Management is an important module of the Smart Parking Management System. It is responsible for managing parking spaces, monitoring their availability, allocating slots to vehicles, and updating parking slot status during vehicle entry and exit operations.

In traditional parking systems, parking spaces are often managed manually, making it difficult to identify available slots and maintain accurate occupancy records. This can result in parking delays, incorrect slot allocation, and inefficient utilization of parking areas.

The proposed Smart Parking Management System uses HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS to provide an organized and user-friendly solution for managing parking spaces. JavaScript handles the application logic, React.js displays parking slot information, and NoSQL stores slot details and their current status.

The module aims to improve parking efficiency, reduce manual errors, and provide accurate information about parking availability.

## 4.4 Checking Parking Availability

Checking Parking Availability is an important module of the Smart Parking Management System. It allows users and administrators to identify vacant parking spaces before assigning a slot to a vehicle.

In traditional parking systems, parking availability is often checked manually, which can lead to delays, confusion, and inefficient utilization of parking spaces. The proposed Smart Parking Management System simplifies this process by providing a computerized method for checking parking slot availability.

The system uses JavaScript as the programming language, React.js for the graphical user interface, and NoSQL for database management. JavaScript retrieves parking information from the database, while React.js displays the availability status in a user-friendly format.

This module helps reduce parking search time, prevent incorrect slot allocation, and improve overall parking management efficiency.

## 4.5 Booking / Slot Allocation

Booking and Slot Allocation is an important module of the Smart Parking Management System. It allows users or parking administrators to reserve and assign available parking spaces to vehicles in an organized and efficient manner.

In traditional parking management systems, parking slots are often allocated manually, which may lead to delays, incorrect assignments, double bookings, and inefficient utilization of parking spaces.

The proposed Smart Parking Management System uses JavaScript as the programming language, React.js for the graphical user interface, and NoSQL for database management.

JavaScript handles booking validation and slot allocation logic, React.js provides an interactive interface for selecting parking spaces, and NoSQL stores booking details and parking slot information.

The main purpose of this module is to simplify parking reservations, prevent duplicate slot allocation, and improve the overall efficiency of parking management operations.

## 4.6 Vehicle Entry, Exit and Payment

Vehicle Entry, Exit and Payment Management is an important module of the Smart Parking Management System. It is responsible for recording vehicle arrival and departure times, managing parking slot occupancy, calculating parking charges, and maintaining payment records.

In traditional parking systems, vehicle entry and exit information is often recorded manually, which can lead to incorrect timestamps, inaccurate parking fee calculations, delays, and difficulties in maintaining parking records.

The proposed Smart Parking Management System uses JavaScript as the programming language, React.js for the graphical user interface, and NoSQL for database management.

JavaScript handles vehicle entry and exit operations, parking duration calculations, and payment processing logic. React.js provides an interactive interface for users and administrators, while NoSQL stores vehicle details, parking transactions, and payment information.

The main purpose of this module is to automate parking operations, reduce manual errors, and ensure accurate parking fee calculation and record management.

## 4.7 Administration and Testing

Administration and Testing are important components of the Smart Parking Management System. The Administration module is responsible for managing parking operations, monitoring vehicle records, controlling parking slot availability, managing users, and maintaining system information. Testing ensures that the application functions correctly, reliably, securely, and efficiently.

The proposed Smart Parking Management System uses JavaScript as the programming language, React.js for the graphical user interface, and NoSQL for database management.

JavaScript handles administrative operations and application logic, React.js provides an interactive administrative dashboard, and NoSQL stores parking records, user information, vehicle details, and payment transactions.

The main purpose of Administration and Testing is to ensure smooth parking operations, maintain accurate information, prevent unauthorized access, and identify errors before the system is deployed.

# 5. SYSTEM DESIGN

System Design is an important phase in the development of the Smart Parking Management System. It involves planning the architecture, components, database structure, user interface, and overall working of the system.

The Smart Parking Management System is designed to automate parking operations such as vehicle registration, parking slot allocation, booking management, vehicle entry and exit, payment calculation, and parking availability tracking.

The system is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS to provide a user-friendly, efficient, and reliable parking management solution.

## 5.1 Module Description

The Smart Parking Management System is divided into several functional modules to ensure efficient management of parking operations. Each module performs specific tasks and interacts with other modules to provide a complete parking management solution.

The system is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS to automate activities such as user registration, vehicle management, parking slot allocation, booking, vehicle entry and exit, payment calculation, and administrative operations.

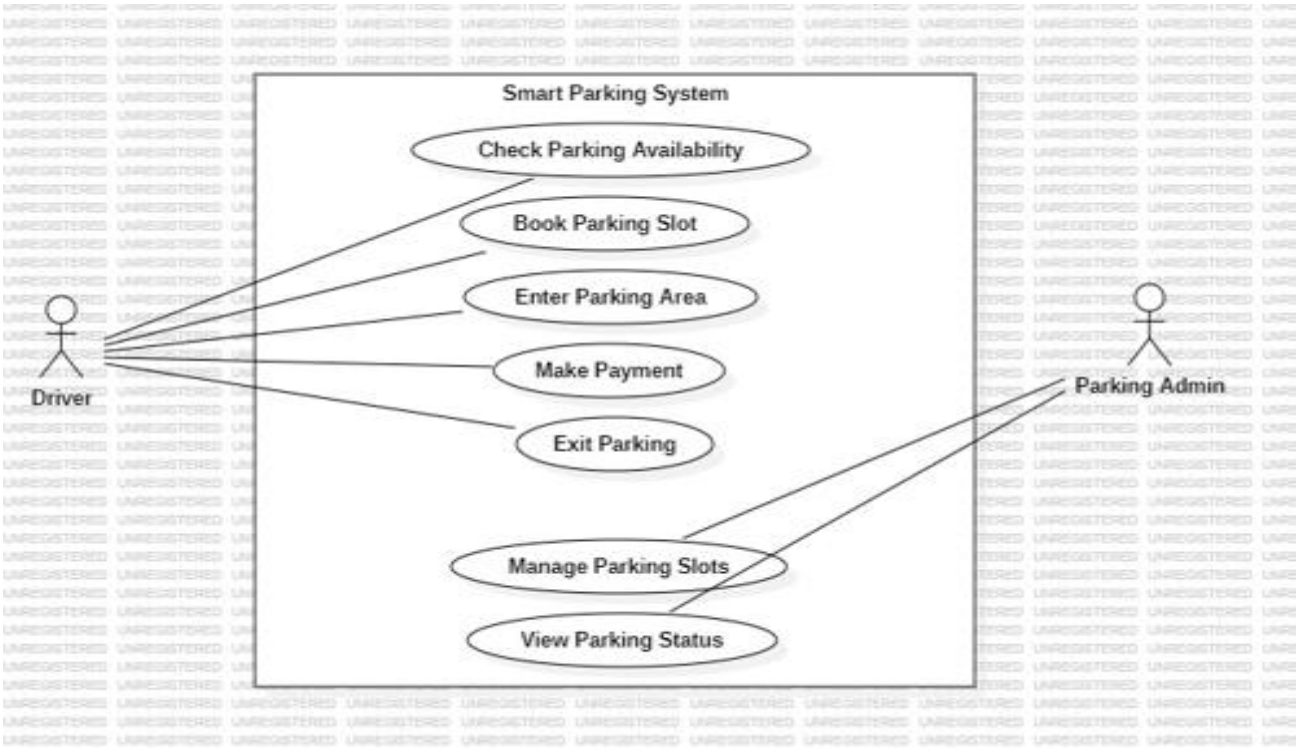
The modular design makes the system easier to develop, test, maintain, and upgrade. It also improves system performance, reliability, and user experience.

## 5.2 Use Case Diagram

A Use Case Diagram is a UML (Unified Modeling Language) diagram that represents the interaction between users and the Smart Parking Management System.

It identifies the different actors, system functionalities, and relationships between them.

The main purpose of the use case diagram is to provide a clear understanding of how users and administrators interact with the system.



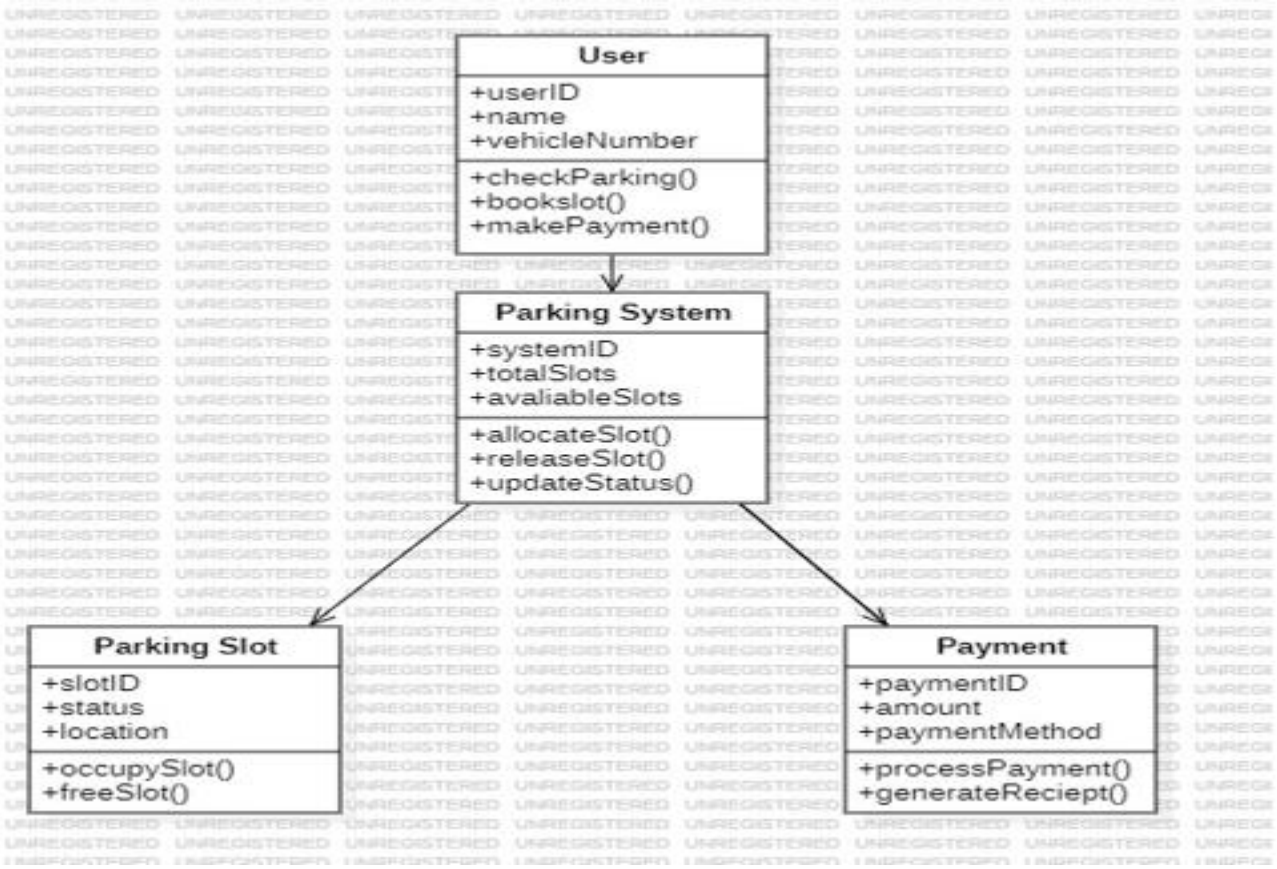

• *Fig. 5.1 Use Case Diagram*

## 5.3 Class Diagram

A Class Diagram is a UML (Unified Modeling Language) structural diagram that represents the classes, attributes, methods, and relationships within a software system.

The Class Diagram for the Smart Parking Management System describes how different components interact to manage user registration, vehicle details, parking slots, bookings, vehicle entry and exit, payments, and administrative operations.

The system is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS.



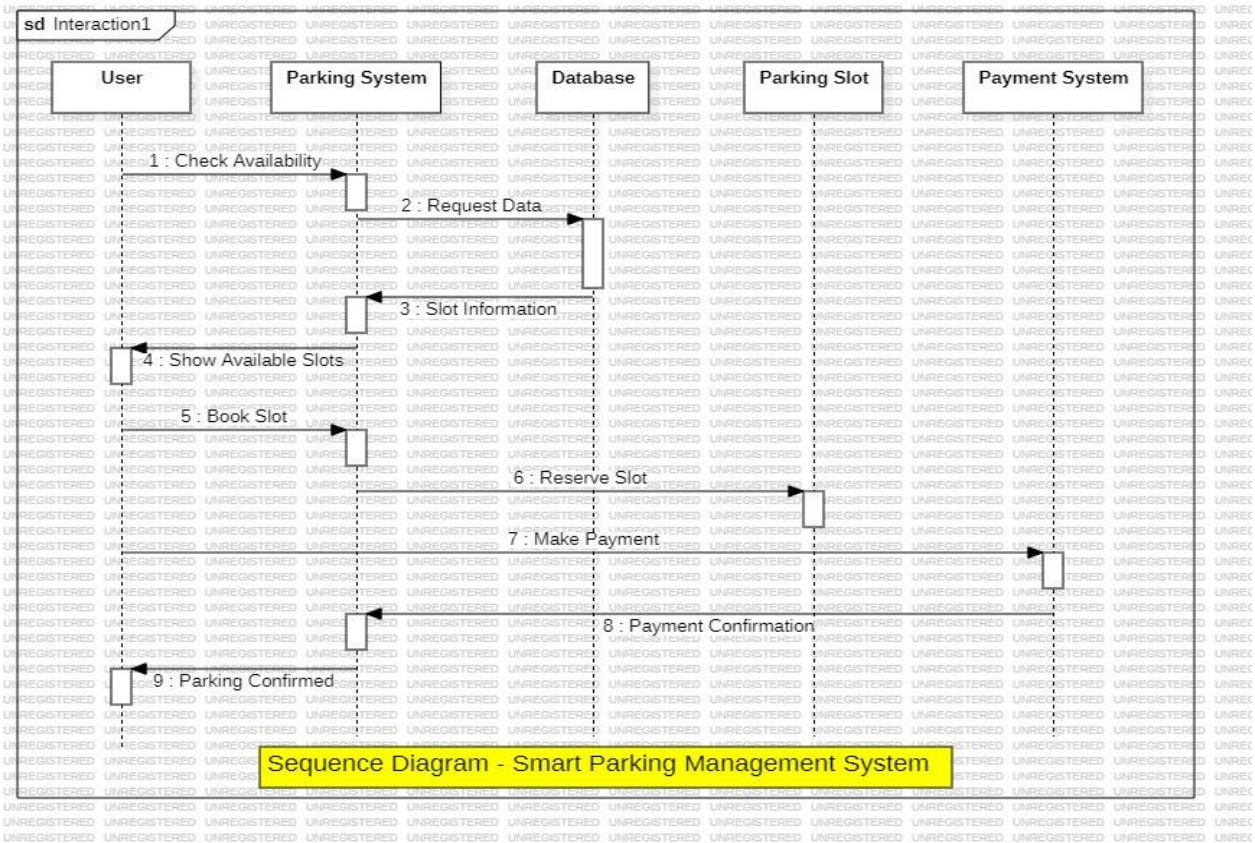

• *Fig. 5.2 Class Diagram*

## 5.4 Sequence Diagram

A Sequence Diagram is a UML (Unified Modeling Language) behavioral diagram that represents the interaction between different objects or components of a system in chronological order.

The Sequence Diagram for the Smart Parking Management System illustrates how users interact with the system to log in, check parking availability, book parking slots, record vehicle entry and exit, calculate parking charges, and complete payments.

The system is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS.




• *Fig. 5.3 Sequence Diagram*

## 5.5 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It shows the interaction between external entities, processes, data stores, and the flow of information.

The Smart Parking Management System uses a DFD to illustrate how user information, vehicle details, parking slot availability, booking records, and payment information are processed and stored.

The system is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS.

## 5.5.1 DFD Level 0

The Level 0 Data Flow Diagram (DFD), also known as the Context Diagram, represents the entire Smart Parking Management System as a single process. It shows how external entities interact with the system through the exchange of information.

The main external entities are:

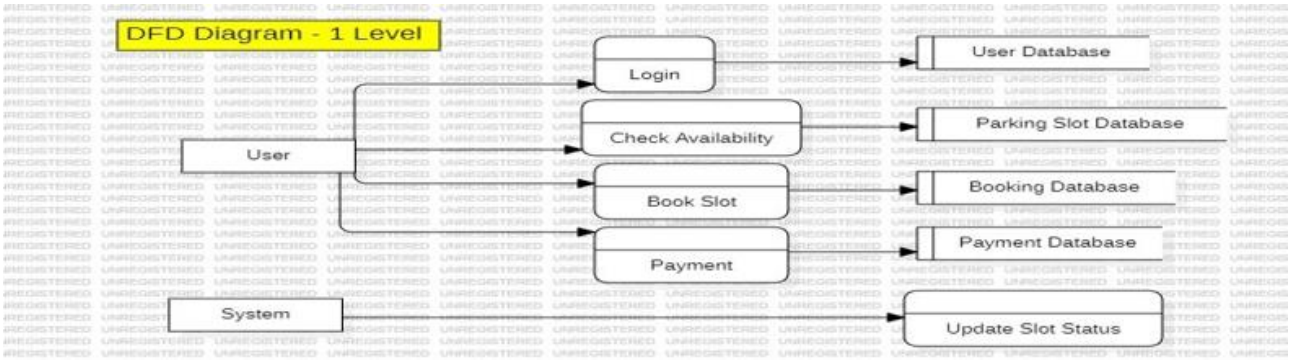
1. User / Vehicle Owner: Registers, logs in, checks parking availability, books parking slots, and receives booking and payment details.
2. Administrator: Manages users, parking slots, vehicle records, payment information, and reports.




• *Fig. 5.4 DFD Level 0*

## 5.5.2 DFD Level 1

The Level 1 Data Flow Diagram (DFD) of the Smart Parking Management System provides a detailed representation of the major processes involved in parking operations. It breaks down the Level 0 context diagram into smaller functional processes and shows how data flows between users, administrators, processes, and databases.

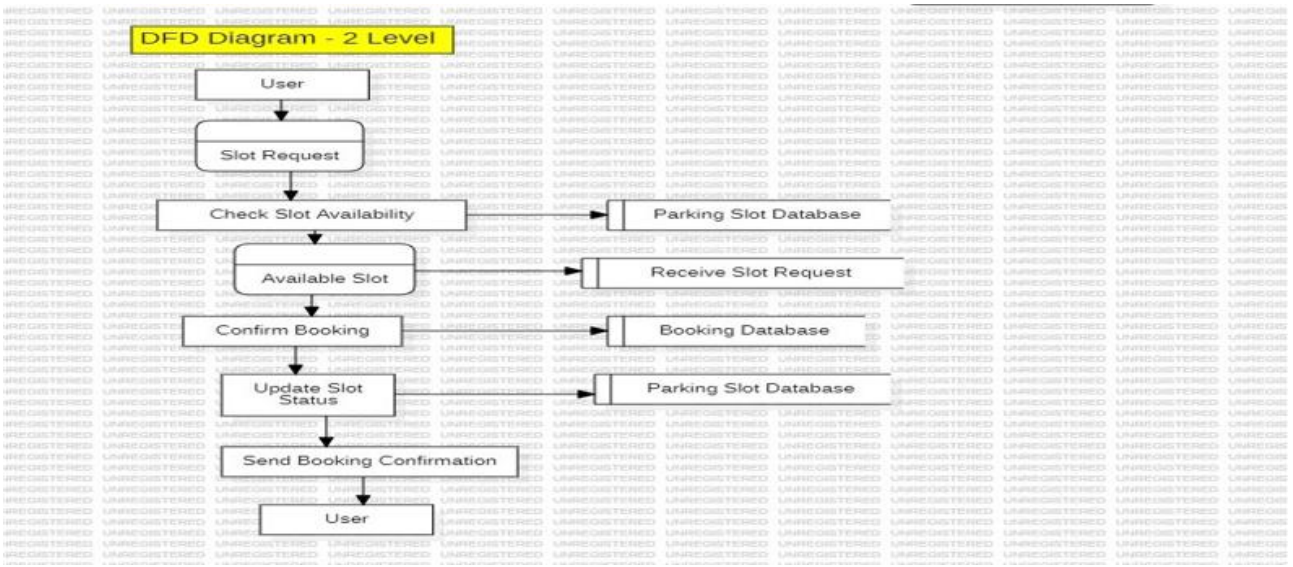



• *Fig. 5.5 DFD Level 1*

### 5.5.3 DFD Level 2

The Level 2 Data Flow Diagram (DFD) of the Smart Parking Management System provides a detailed representation of the internal processes involved in parking slot booking and reservation management.

It further decomposes the 3.0 Booking Management process from the Level 1 DFD into smaller sub processes, showing how data flows between the user, booking operations, and database.

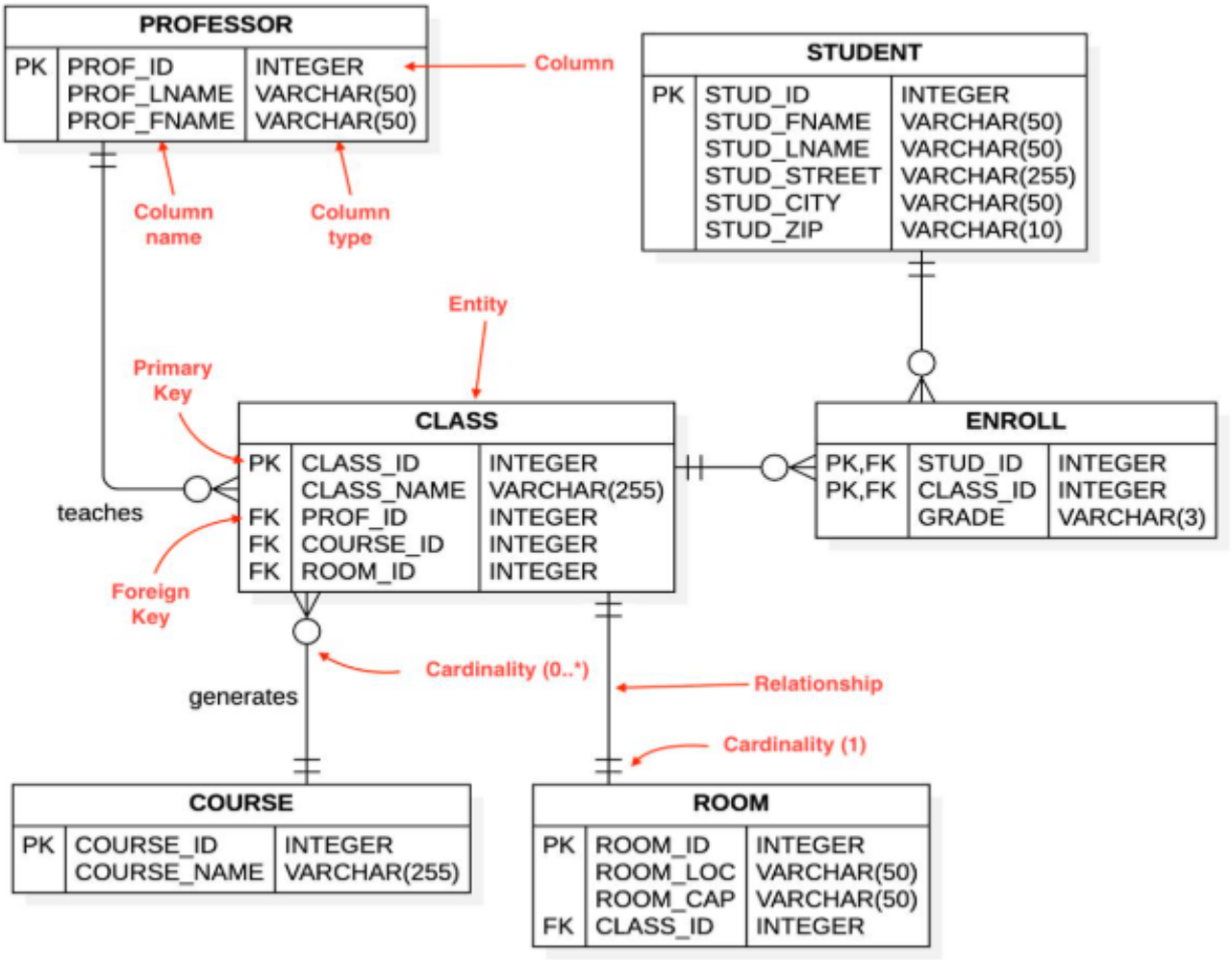



• *Fig. 5.6 DFD Level 2*

## 5.6 Entity–Relationship Diagram

An Entity–Relationship Diagram (ER Diagram) is a graphical representation of the database structure of a software system. It illustrates the entities, attributes, primary keys, foreign keys, and relationships between different database tables.

The ER Diagram of the Smart Parking Management System represents how user information, vehicle details, parking slots, bookings, vehicle entry and exit records, and payment transactions are stored and connected.




• *Fig. 5.7 Entity–Relationship Diagram*

## 5.7 Data Dictionary

A Data Dictionary is a structured collection of information that describes the data elements, database tables, attributes, data types, constraints, and relationships used in a software system.

The Data Dictionary for the Smart Parking Management System provides detailed information about the database structure used to store and manage user details, vehicle information, parking slots, bookings, vehicle entry and exit records, payments, and administrator information.

The data dictionary helps developers understand the database design and ensures accuracy, consistency, and efficient data management.

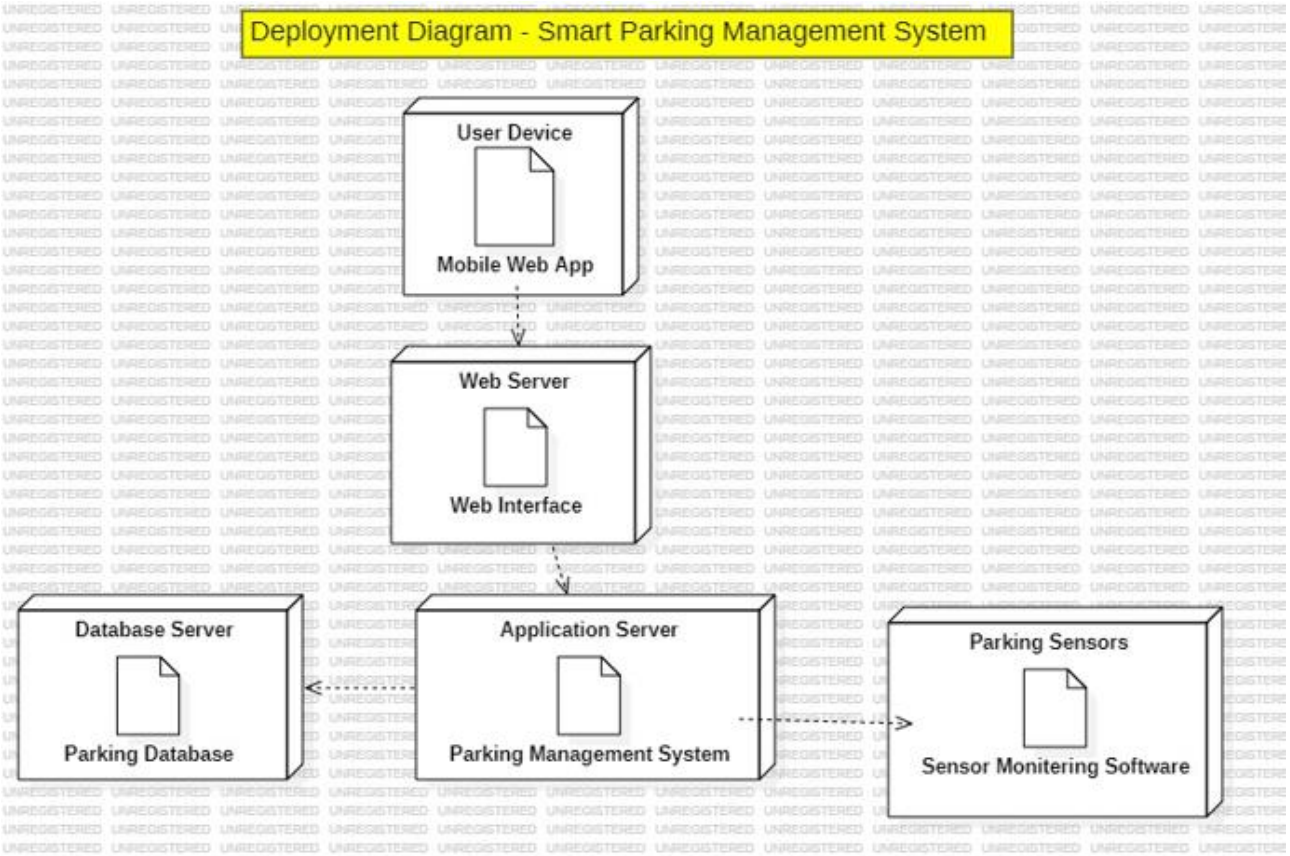
| Field example | Meaning | Notes |
|---|---|---|
| slot_id | Unique slot identifier | Primary key; verify schema |
| slot_number | Readable slot label | Use actual type |
| status | Recorded slot state | Use code's values |
| booking_id / user_id | Optional references | Only if those modules exist |

## 5.8 Deployment Diagram

A Deployment Diagram is a UML (Unified Modeling Language) structural diagram that represents the physical architecture of a software system. It shows how software components are deployed on hardware devices and how different nodes communicate with each other.

The Deployment Diagram of the Smart Parking Management System illustrates the deployment of the JavaScript application, React.js graphical user interface, NoSQL database, and other system components.

The system uses HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS to manage parking slot allocation, vehicle registration, booking, vehicle entry and exit, payment calculation, and administrative operations.

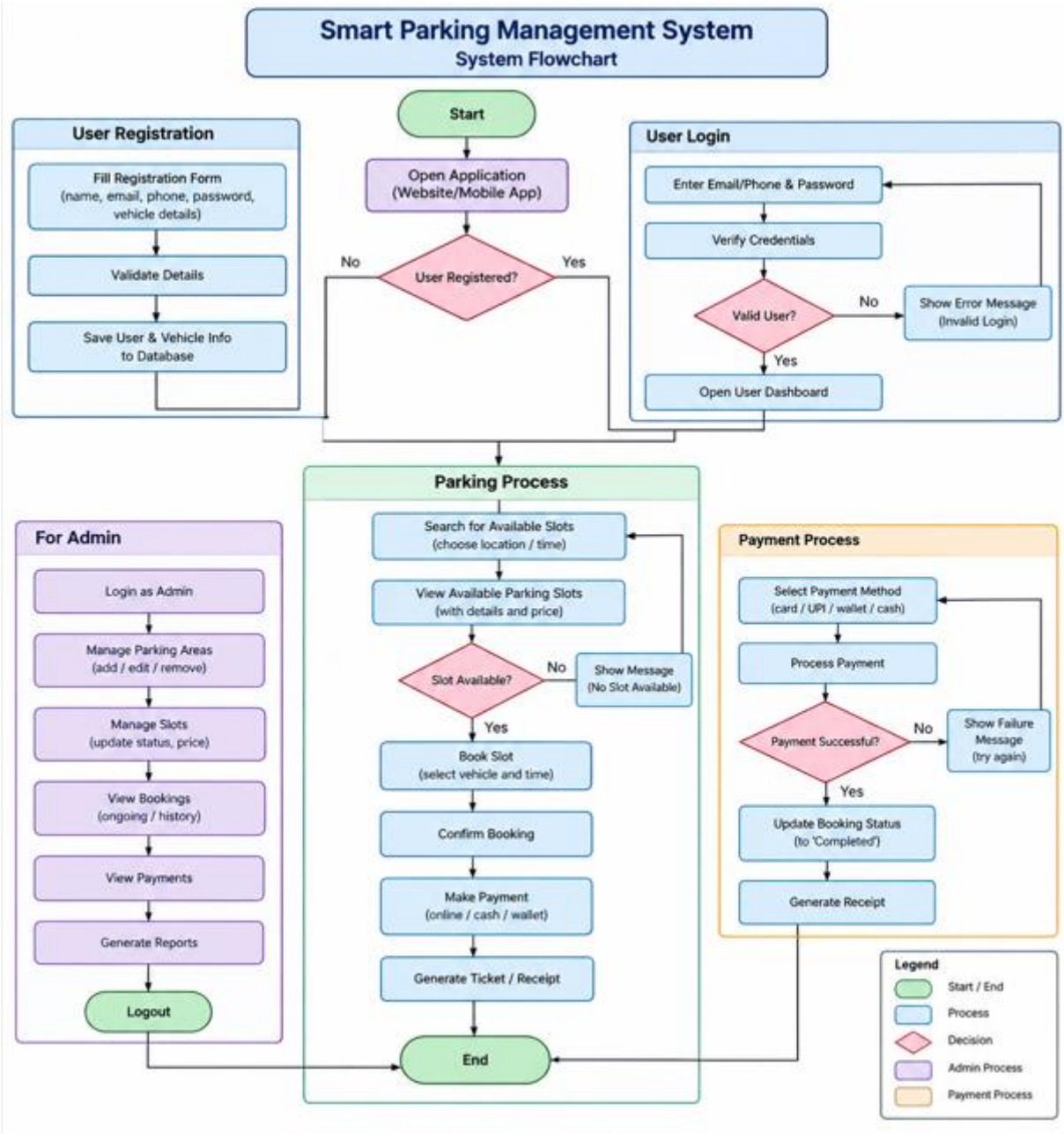



• *Fig. 5.8 Deployment Diagram*

## 5.9 System Flowchart

A System Flowchart is a graphical representation of the sequence of operations performed within a software system. It illustrates the flow of information, decision-making processes, and interactions between different system components.

The System Flowchart of the Smart Parking Management System explains the complete working process, including user registration, login, parking slot availability checking, slot booking, vehicle entry, parking fee calculation, payment processing, and vehicle exit.



# 6. IMPLEMENTATION AND TESTING

Implementation and Testing are important phases in the Software Development Life Cycle (SDLC). The implementation phase involves converting the system design into a functional software application, while the testing phase ensures that the developed system works correctly, efficiently, and reliably.

The Smart Parking Management System is designed and implemented using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. The system automates parking operations such as user registration, login, vehicle management, parking slot allocation, booking management, vehicle entry and exit, payment calculation, and report generation.

The main objective of implementation and testing is to develop a reliable, secure, and user-friendly application that reduces manual parking management activities and improves parking efficiency.

## 6.1 Key Code Snippets

The Smart Parking Management System is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. The system provides functionalities such as user registration, login authentication, vehicle registration, parking slot management, booking, vehicle entry and exit, parking fee calculation, payment management, and report generation.

The following key code snippets demonstrate the implementation of important modules of the Smart Parking Management System using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. NoSQL is used for these examples because it is lightweight and suitable for desktop applications.

### 6.1.1 Database Connection and Table Creation

Database Connection and Table Creation are important parts of the implementation of the Smart Parking Management System. The database is responsible for storing, retrieving, updating, and managing information related to users, vehicles, parking slots, bookings, vehicle entry and exit records, payments, and administrators.

The Smart Parking Management System is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. NoSQL is a lightweight relational database that does not require a separate database server, making it suitable for desktop-based applications.

Code:

```
export function haversineDistance(lat1, lng1, lat2, lng2) {

  const R = 6371;

  const dLat = ((lat2 - lat1) * Math.PI) / 180;

  const dLng = ((lng2 - lng1) * Math.PI) / 180;

  const a =

    Math.sin(dLat / 2) ** 2 +

    Math.cos((lat1 * Math.PI) / 180) * Math.cos((lat2 * Math.PI) / 180) *
Math.sin(dLng / 2) ** 2;

  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));

}

export function formatDistance(km) {

  if (km < 1) return `${Math.round(km * 1000)} m`;

  return `${km.toFixed(1)} km`;

}

export function formatTimeRange(start, end) {

  const s = new Date(start);

  const e = new Date(end);
```

```
  const opts = { hour: "numeric", minute: "2-digit" };
  return `${s.toLocaleTimeString([], opts)} – ${e.toLocaleTimeString([], opts)}`;
}
export function formatDateLabel(d) {
  return new Date(d).toLocaleDateString([], { weekday: "short", month: "short", day: "numeric" });
}
export function defaultStartTime() {
  const d = new Date();
  d.setMinutes(d.getMinutes() + 30);
  d.setMinutes(0);
  const tzOffset = d.getTimezoneOffset() * 60000;
  return new Date(d.getTime() - tzOffset).toISOString().slice(0, 16);
}
export function isUpcoming(endTime) {
  return new Date(endTime).getTime() > Date.now();
}
```

### 6.1.2 User Registration

User Registration is an important module of the Smart Parking Management System. It allows new users to create an account by providing personal information such as name, email address, contact number, username, and password.

The User Registration Module is developed using HTML5, CSS3, JavaScript, React.js, Typescript and Tailwind CSS. React.js provides a user-friendly graphical interface, while NoSQL stores registered user information securely.

The main purpose of this module is to maintain user records, provide secure authentication, and allow registered users to access parking services such as parking slot availability, booking, vehicle registration, and payment management.

Code:

```
export default function Register() {

  const [email, setEmail] = useState("");

  const [password, setPassword] = useState("");

  const [confirmPassword, setConfirmPassword] = useState("");

  const [error, setError] = useState("");

  const [loading, setLoading] = useState(false);

  const [showOtp, setShowOtp] = useState(false);

  const [otpCode, setOtpCode] = useState("");

  const handleSubmit = async (e) => {

    e.preventDefault();

    setError("");

    if (password !== confirmPassword) {

      setError("Passwords do not match");

      return;

    }

    setLoading(true);
```

```
  try {
    await base44.auth.register({ email, password });
    setShowOtp(true);
  } catch (err) {
    setError(err.message || "Registration failed");
  } finally {
    setLoading(false);
  }
};
const handleVerify = async () => {
  setError("");
  setLoading(true);
  try {
    const result = await base44.auth.verifyOtp({ email, otpCode });
    if (result?.access_token) {
      base44.auth.setToken(result.access_token);
    }
    window.location.href = safeReturnTo();
  } catch (err) {
    setError(err.message || "Invalid verification code");
  } finally {
    setLoading(false);
  }
};
```

### 6.1.3 Login Validation

Login validation is an important security feature of the Smart Parking Management System. It verifies the credentials entered by users and administrators before granting access to the application.

City Park provides separate login access for Users and Administrators. Login validation ensures that only registered users with valid credentials can access their respective dashboards and perform authorized operations.

Code:

```
export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  // Post-login destination (e.g. the MCP OAuth consent page sends users here
  // with returnTo so the grant flow can resume). Same-origin paths only.
  const returnTo = safeReturnTo();
  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await base44.auth.loginViaEmailPassword(email, password);
      window.location.href = returnTo;
    } catch (err) {
      setError(err.message || "Invalid email or password");
```

```
    } finally {
      setLoading(false);
    }
  };
  const handleGoogle = () => {
    base44.auth.loginWithProvider("google", returnTo);
  };
  return (
    <AuthLayout
      icon={LogIn}
      title="Welcome back"
      subtitle="Log in to your account"
      footer={
        <>
          Don't have an account?{" "}
          <Link
            to={"/register" + (returnTo !== "/" ? "?returnTo=" +
encodeURIComponent(returnTo) : "")}
            className="text-primary font-medium hover:underline"
          >
            Create one
          </Link>
        </>
      }
    >
```

```
    <Button
      variant="outline"
      className="group w-full h-12 text-sm font-semibold mb-6 hover:border-primary/40 hover:bg-primary/5 hover:-translate-y-0.5 hover:shadow-md transition-all"
      onClick={handleGoogle}
    >
      <GoogleIcon className="w-5 h-5 mr-2 transition-transform group-hover:scale-110" />
      Continue with Google
    </Button>

    <div className="relative mb-6">
      <div className="absolute inset-0 flex items-center">
        <div className="w-full border-t border-border" />
      </div>
      <div className="relative flex justify-center text-xs uppercase">
        <span className="bg-card px-3 text-muted-foreground">or</span>
      </div>
    </div>
```

### 6.1.4 Checking Parking Availability

Checking parking availability is one of the major features of the Smart Parking Management System. It allows users to view the current status of parking slots and identify available spaces before making a booking.

City Park provides parking slot information through its user interface. The system displays whether a parking slot is available, booked, or occupied. This helps users select a suitable parking space and reduces the time spent searching for parking.

## 6.1.5 Creating a Booking / Slot Allocation

Creating a booking and slot allocation is one of the core features of the Smart Parking Management System. It allows users to reserve an available parking space for their vehicles through the City Park application.

The system enables users to select an available parking slot, enter the required booking details, and confirm their reservation. Once the booking is successfully completed, the selected parking slot is allocated to that user and its availability status is updated in the system.

This process helps avoid confusion, reduces parking conflicts, and ensures proper utilization of available parking spaces.

Code:

```
function BookingCard({ booking, active }) {

  const qrData = JSON.stringify({

    id: booking.id,

    lot: booking.lot_name,

    spot: booking.spot_number,

    start: booking.start_time,

    end: booking.end_time,

  });

  const qrUrl = `https://api.qrserver.com/v1/create-qr-
code/?size=160x160&data=${encodeURIComponent(qrData)}`;

  return (

    <div

      className={`relative overflow-hidden bg-card rounded-xl border p-4 transition-
all hover:-translate-y-0.5 hover:shadow-md ${

        active

          ? "border-primary/40 shadow-md shadow-primary/15 bg-gradient-to-br from-
primary/5 via-transparent to-[#8B5CF6]/10"
```

```
        : "border-border"
      }`}
    >
      {active && (
        <div
          className="absolute inset-x-0 top-0 h-1 animate-gradient-pan"
          style={{
            backgroundImage: "linear-gradient(90deg, #4F46E5, #8B5CF6, #06B6D4, #4F46E5)",
            backgroundSize: "220% 100%",
          }}
        />
      )}
      <div className="flex gap-4">
        <div
          className={`w-20 h-20 rounded-lg bg-white flex-shrink-0 flex items-center justify-center overflow-hidden transition-all ${
            active ? "ring-2 ring-primary/30 shadow-lg shadow-primary/15" : "ring-1 ring-border"
          }`}
        >
          <img src={qrUrl} alt="Booking QR" className="w-full h-full object-contain p-1" />
        </div>
        <div className="flex-1 min-w-0">
          <div className="flex items-start justify-between gap-2">
```

```
        <h3 className="font-heading font-bold text-foreground text-sm truncate">{booking.lot_name}</h3>

        <span

          className={`text-xs font-semibold px-2 py-0.5 rounded-full whitespace-nowrap ${

            booking.status === "confirmed"

              ? "bg-emerald-50 dark:bg-emerald-500/15 text-emerald-600 dark:text-emerald-400"

              : booking.status === "completed"

              ? "bg-muted text-muted-foreground"

              : "bg-red-50 dark:bg-red-500/15 text-red-600 dark:text-red-400"

          }`

}
```

### 6.1.6 Updating Slot Status

Updating parking slot status is an important feature of the Smart Parking Management System. It ensures that the availability information of parking spaces remains accurate and up to date.

City Park allows the system to update the status of parking slots based on booking activities and administrative operations. When a user successfully books a parking slot, its status is changed from Available to Booked. Similarly, when a parking session is completed or a booking is cancelled, the slot can be made available again according to the parking and booking rules.

This feature helps administrators manage parking spaces efficiently and allows users to view the latest slot availability.

Code:

```
export default function BookingSuccess() {

  const navigate = useNavigate();

  const [booking, setBooking] = useState(null);

  const [error, setError] = useState(null);

  const [loading, setLoading] = useState(true);

  useEffect(() => {

    const params = new URLSearchParams(window.location.search);

    const sessionId = params.get("session_id");

    if (!sessionId) {

      setError("Missing payment session.");

      setLoading(false);

      return;

    }

    base44.functions

      .invoke("verifyPayment", { sessionId })
```

```
      .then((res) => {
        if (res.data?.booking) {
          setBooking(res.data.booking);
        } else {
          setError(res.data?.error || "Could not verify payment");
        }
      })
      .catch((e) => setError(e.message || "Verification failed"))
      .finally(() => setLoading(false));
  }, []);

  if (loading) {
    return (
      <div className="flex-1 flex items-center justify-center">
        <div className="text-center">
          <Loader2 className="w-8 h-8 text-primary animate-spin mx-auto" />
          <p className="text-sm text-muted-foreground mt-3">Confirming your reservation…</p>
        </div>
      </div>
    );
  }
```

### 6.1.7 Vehicle Session / Payment Record

Vehicle Session and Payment Record Management is an important feature of the Smart Parking Management System. It helps maintain information about vehicles using parking spaces and records the payments made by users for parking services.

City Park maintains booking and payment-related information so that users can view their parking details and administrators can monitor parking transactions. The vehicle session record helps track the parking duration, while the payment record maintains details of the amount paid and transaction status.

This feature improves transparency, simplifies parking fee management, and helps maintain accurate transaction records.

## 6.2 User Interface Screenshots

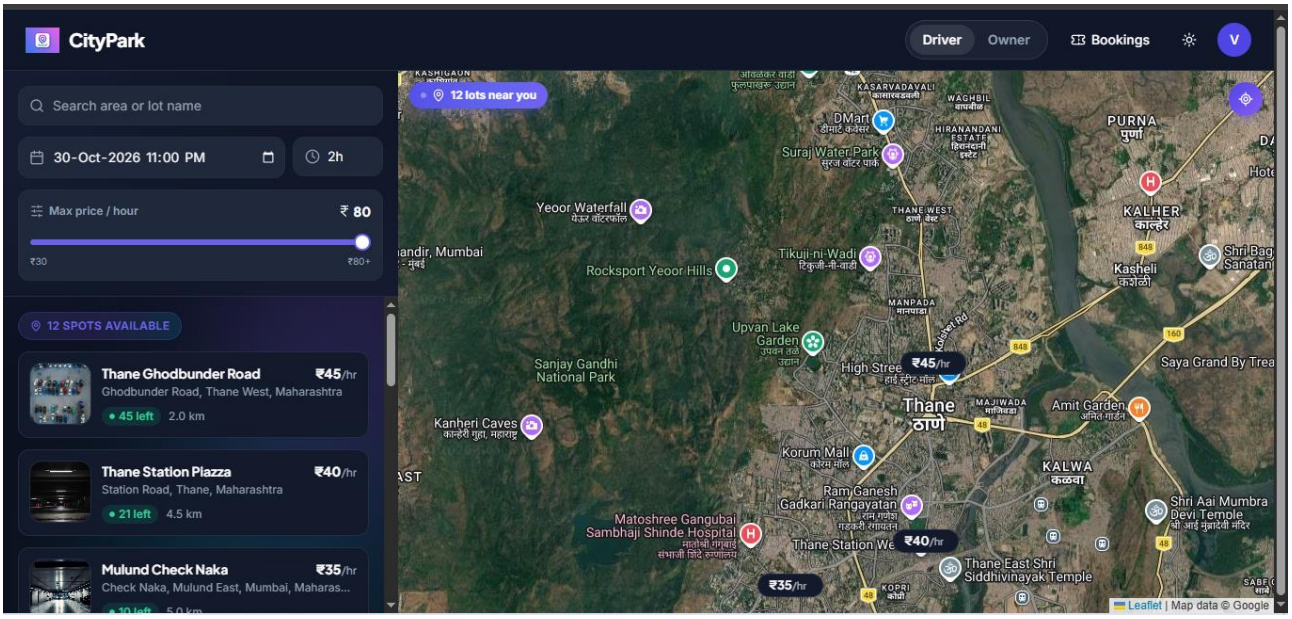



*Fig. 6.1 Main Screen*

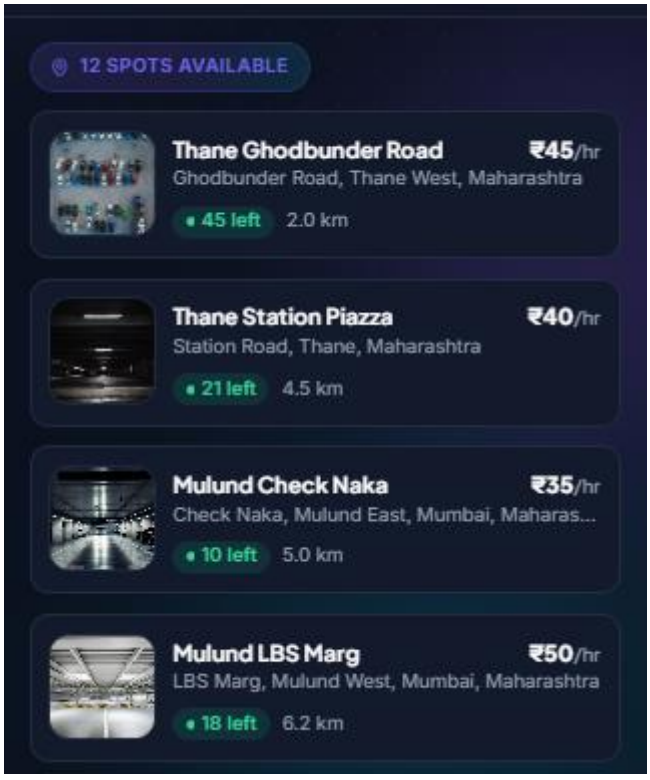



*Fig. 6.2 Parking Slot Dashboard*

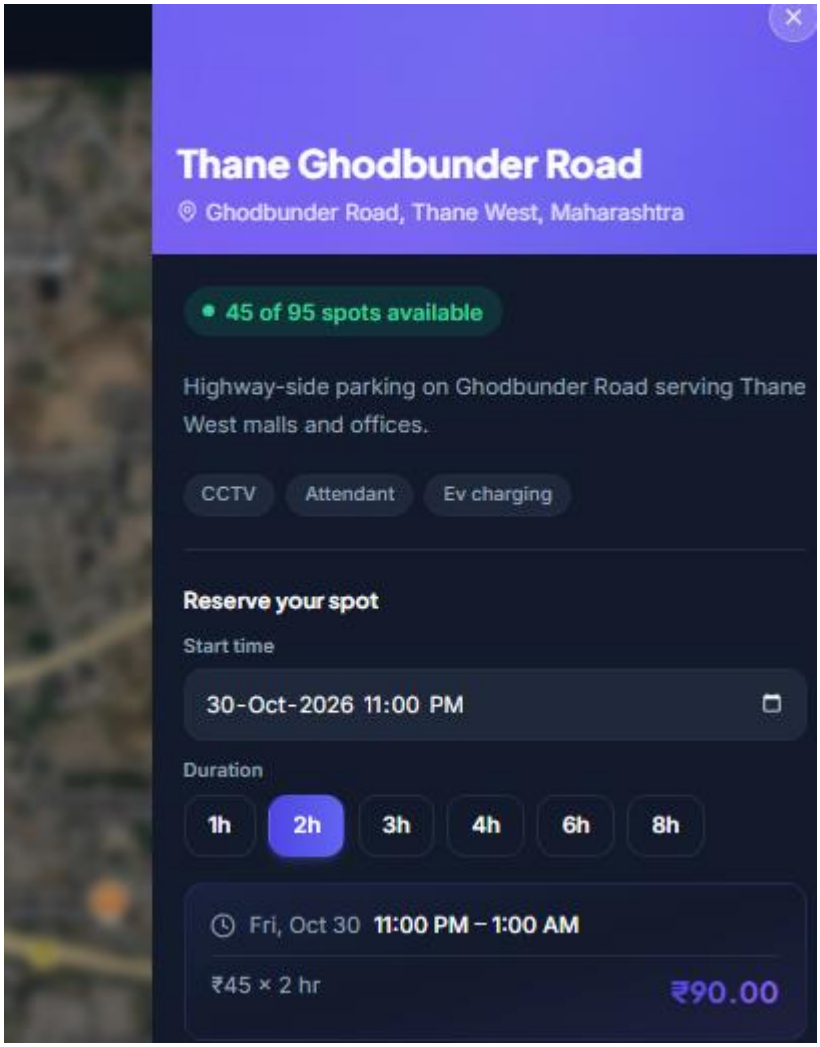



*Fig. 6.3 Availability Result*




*Fig. 6.4 Slot Management*




*Fig. 6.5 Booking Form*

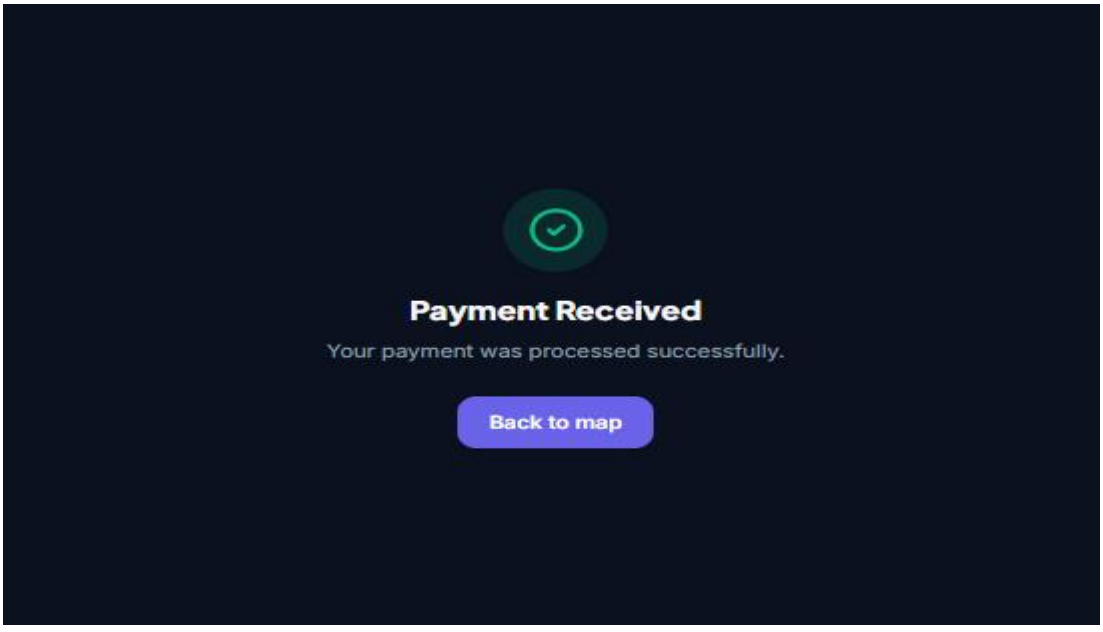



*Fig. 6.6 Booking Confirmation*

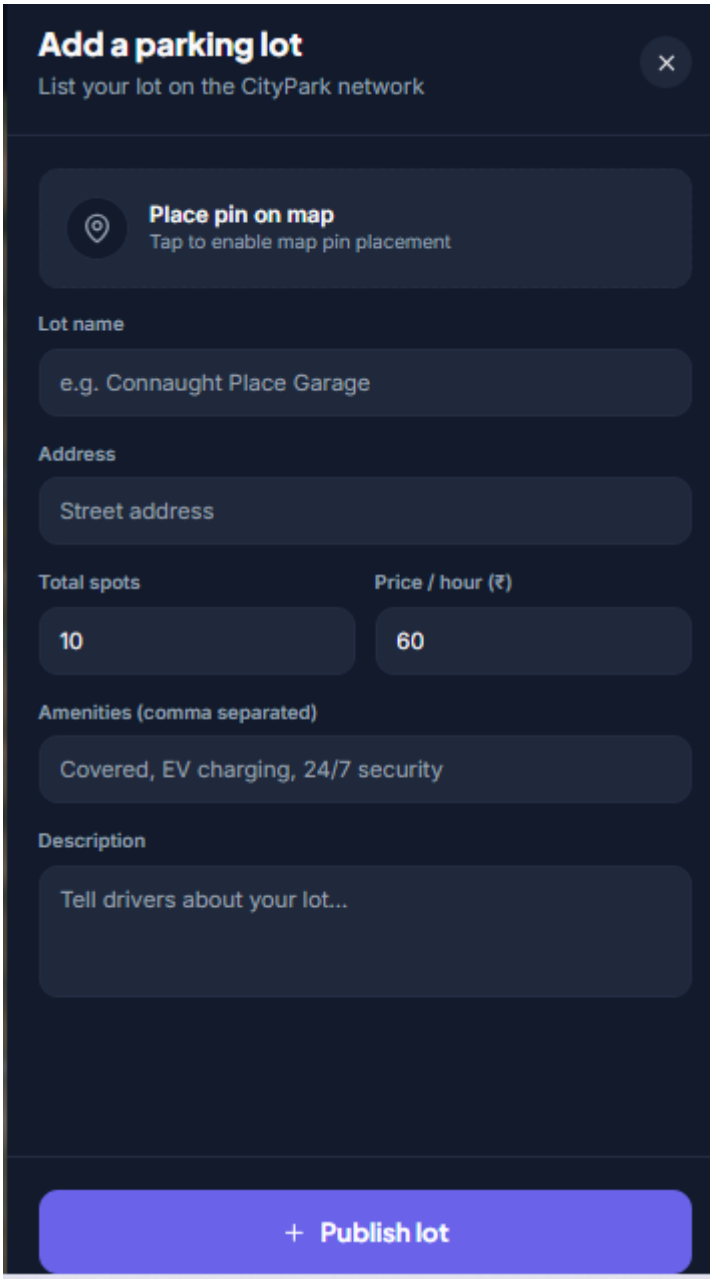



*Fig. 6.7 Administrator Dashboard*

| | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | lot_id | lot_name | lot_addre | start_time | end_time | duration_ | total_pric | status | spot_num | payment_ | id | created_d | updated_ | created_b | is_sample |
| 2 | 6ab69e57 | Thane Gh | Ghodbunc | 2026-10-3 | 2026-10-3 | 1 | 45 | confirmec | 1 | upi_17933 | 6ae49734 | 2026-10-3 | 2026-10-3 | 6ae496b1 | FALSE |
| 3 | 6ab69e57 | Mulund Cl | Check Nak | 2026-09-2 | 2026-09-2 | 2 | 70 | confirmec | 1 | upi_17905 | 6aba0f99a | 2026-09-2 | 2026-09-2 | 6ab66ad1 | FALSE |
| 4 | 6ab69e57 | Thane Gh | Ghodbunc | 2026-09-2 | 2026-09-2 | 1 | 45 | confirmec | 1 | upi_17903 | 6ab74195 | 2026-09-2 | 2026-09-2 | 6ab73105 | FALSE |
| 5 | 6ab69e57 | Vashi Sect | Sector 17, | 2026-09-2 | 2026-09-2 | 1 | 40 | confirmec | 1 | upi_17903 | 6ab73d26 | 2026-09-2 | 2026-09-2 | 6ab73c8b | FALSE |
| 6 | 6ab69e57 | Mulund Cl | Check Nak | 2026-09-2 | 2026-09-2 | 1 | 35 | confirmec | 1 | upi_17903 | 6ab729a3 | 2026-09-2 | 2026-09-2 | 6ab7291e | FALSE |
| 7 | 6ab69e57 | Mulund Cl | Check Nak | 2026-09-2 | 2026-09-2 | 1 | 35 | confirmec | 1 | upi_17903 | 6ab72811 | 2026-09-2 | 2026-09-2 | 6ab66ad1 | FALSE |
| 8 | 6ab69e57 | Mulund LE | LBS Marg, | 2026-09-2 | 2026-09-2 | 1 | 50 | confirmec | 1 | upi_17903 | 6ab6c7f56 | 2026-09-2 | 2026-09-2 | 6ab66ad1 | FALSE |
| 9 | 6ab69e57 | Mulund Cl | Check Nak | 2026-09-2 | 2026-09-2 | 1 | 35 | confirmec | 1 | upi_17903 | 6ab6c4dd | 2026-09-2 | 2026-09-2 | 6ab66ad1 | FALSE |
| 10 | 6ab69e57 | Thane Sta | Station Rc | 2026-09-2 | 2026-09-2 | 1 | 40 | confirmec | 1 | upi_17903 | 6ab6c441 | 2026-09-2 | 2026-09-2 | 6ab66ad1 | FALSE |
| 11 | 6ab69e57 | Thane Gh | Ghodbunc | 2026-09-2 | 2026-09-2 | 1 | 45 | confirmec | 1 | upi_17903 | 6ab6c422 | 2026-09-2 | 2026-09-2 | 6ab66ad1 | FALSE |

*Fig. 6.8 Database Records*



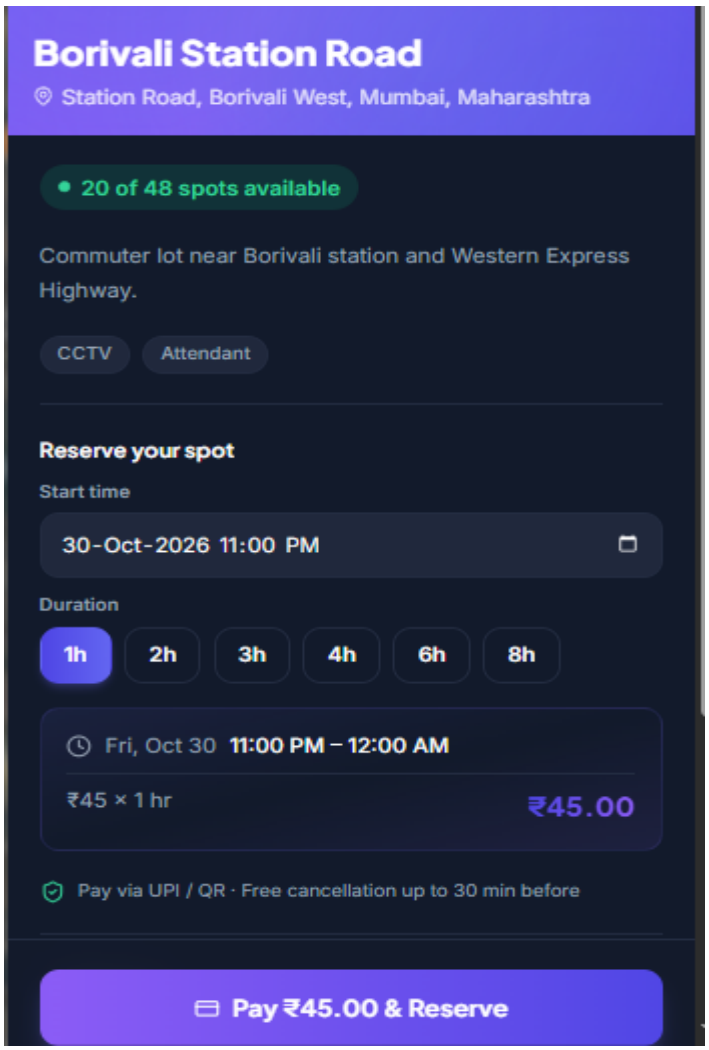




*Fig. 6.9 Validation*

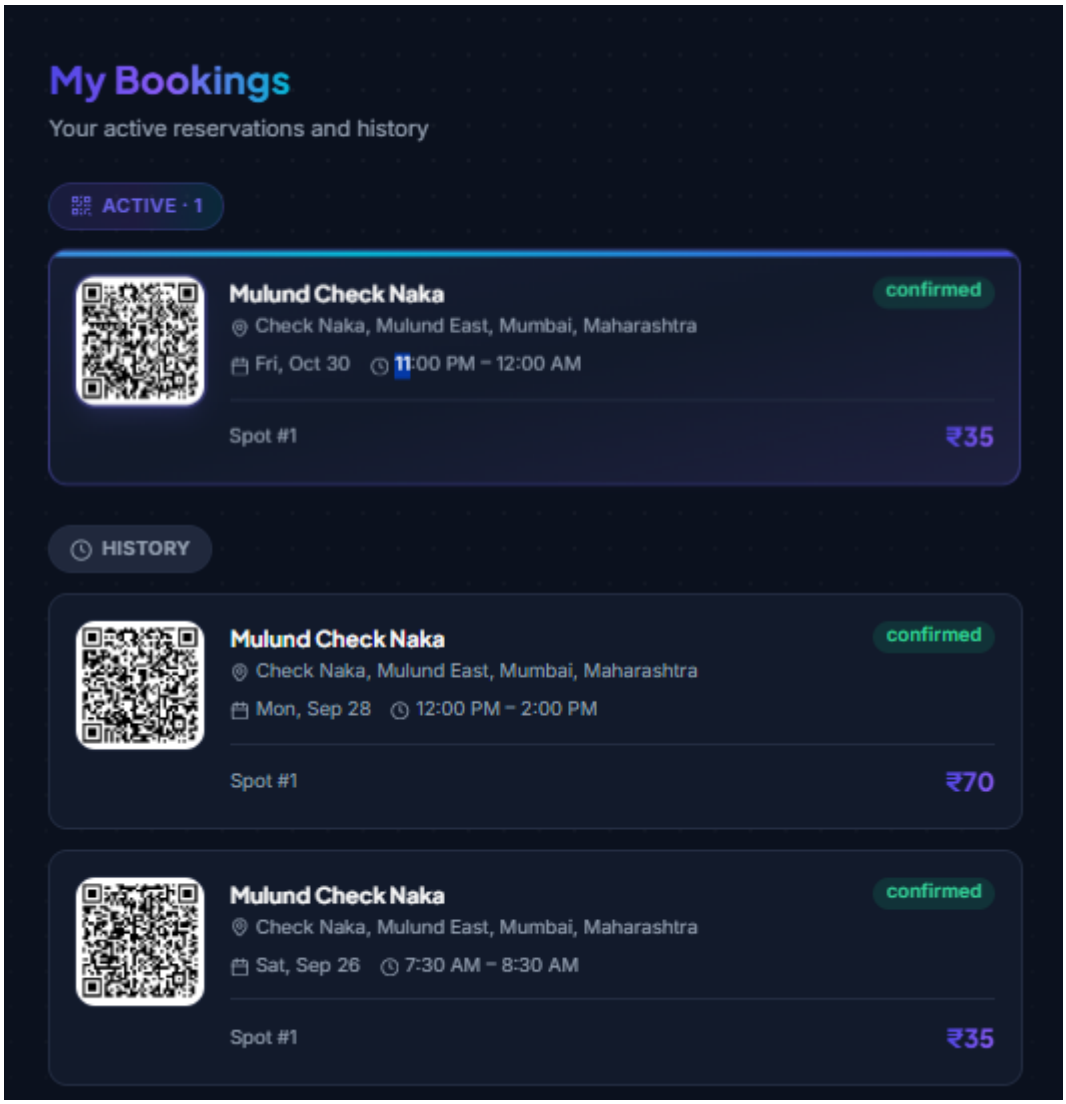



*Fig. 6.10 Test Output.*

## 6.3 Testing Strategy

A test strategy is a systematic approach used to verify and validate the functionality, performance, security, and reliability of a software application. It defines the testing methods, techniques, and activities required to ensure that the developed system meets the specified requirements.

For the Smart Parking Management System (City Park), testing is performed to ensure that users can register, log in, check parking availability, book parking slots, and make payments successfully. It also verifies that administrators can manage parking slots, monitor bookings, and maintain parking records.

The main objective of the testing strategy is to identify errors, improve application quality, and ensure that the system works correctly.

The main objectives of testing City Park are:

1. To verify that all application modules function correctly.
2. To ensure that user registration and login work properly.
3. To verify the accuracy of parking slot availability.
4. To ensure that users can book available parking slots.
5. To check that parking slot status is updated after booking or cancellation.
6. To verify payment processing and payment record management.
7. To ensure that administrators can manage parking information.
8. To identify and correct errors before final deployment.
9. To verify that the application is compatible with different devices and browsers.
10. To ensure that user information is protected from unauthorized access.

### 6.3.1 Test Case Matrix

TC-01 Launch application: main screen opens.

TC-02 Load slots: stored records appear.

TC-03 Check status: recorded status is displayed.

TC-04 Invalid input: validation appears and invalid data is not saved.

TC-05 Database unavailable: safe error handling.

TC-06 Restart: records persist if database-backed.

TC-07 Book available slot: confirmation appears if booking exists.

TC-08 Book unavailable slot: conflict is rejected if booking exists.

TC-09 Admin-only update: unauthorized action is denied if roles exist.

TC-10 Payment: fee/status is correct if implemented.

Mark each Pass, Fail or Not Run based on actual evidence.

| Test ID | Scenario | Expected result | Actual status |
|---|---|---|---|
| TC-01 | Launch application | Main screen opens | Pass |
| TC-02 | Load slots | Stored records appear | Pass |
| TC-03 | Check status | Recorded status appears | Pass |
| TC-04 | Invalid input | Validation shown | Pass |
| Test ID | Scenario | Expected result | Actual status |
| TC-05 | Database offline | Safe error handling | Pass |
| TC-06 | Restart app | Records persist if DB-backed | Not run |
| TC-07 | Booking conflict | Rejected if booking exists | Not run |

# 7. CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

The Smart Parking Management System (City Park) is a web-based application developed to simplify and improve the traditional parking management process. The main objective of this project is to provide an efficient platform where users can check parking slot availability, book parking spaces, and manage parking-related activities through an online system.

The project provides separate interfaces for users and administrators, allowing both to perform their respective operations. Users can register, log in, view available parking slots, make bookings, and access their booking and payment information. Administrators can manage parking slots, monitor bookings, update parking information, and maintain parking records.

The development of City Park demonstrates how modern web technologies can be used to solve common parking-related problems. The application reduces the dependency on manual parking management, improves the organization of parking information, and provides a convenient way to manage parking reservations.

During the development of this project, important software engineering concepts such as requirement analysis, system design, database management, user authentication, module development, and software testing were considered. The project also helped in understanding the practical implementation of a web-based application and the integration of different software modules.

## 7.2 Limitations

Although the Smart Parking Management System (City Park) provides an efficient way to manage parking slots, bookings, and payment-related information, the application has certain limitations. These limitations identify the areas where the system may require further improvements and additional functionality.

The major limitations of City Park are explained below:

**1. Internet Dependency**

- The application requires a stable internet connection to access its features.
- Users may face difficulties accessing the system in areas with poor network connectivity.
- Online booking and database operations may be interrupted due to internet problems.

**2. No Automatic Vehicle Detection**

- The system may not automatically detect the entry and exit of vehicles.
- Without sensors or camera-based technology, vehicle movement must be recorded manually or through application operations.
- This can affect the accuracy of real-time parking slot information.

**3. Limited Real-Time Availability**

- Parking slot availability depends on the accuracy and frequency of database updates.
- If slot information is not updated immediately, users may see outdated availability.
- The system may not automatically identify whether a vehicle has physically occupied or vacated a parking space.

**4. Payment Gateway Dependency**

- Online payment functionality depends on the availability of the integrated payment service.
- Failed transactions or network interruptions may affect payment processing.
- Payment confirmation may be delayed due to external payment service issues

**5. Limited Scalability**

- The system may require additional resources and modifications to support a large number of users and parking locations.
- Managing a large volume of bookings and transactions may require database and performance optimization.
- Additional infrastructure may be necessary when expanding the application.

**6. Dependence on Accurate Data**

- The system depends on users and administrators entering correct information.
- Incorrect vehicle numbers, booking details, or parking slot information may create operational problems.
- Regular updates and monitoring are necessary to maintain data accuracy.

**7. Limited Physical Parking Management**

- The application manages parking information digitally but may not control physical parking barriers or gates.
- It may not automatically guide vehicles to specific parking spaces.
- Physical security and vehicle monitoring may require separate hardware systems.

**8. Device and Browser Compatibility**

- Application performance may vary depending on the user's device, browser, and internet speed.
- Older browsers or devices may not support all application features properly.
- Regular compatibility testing is required.

**9. Security and Privacy Concerns**

- User accounts and vehicle information require proper protection.
- Unauthorized access may occur if authentication and access controls are not configured correctly.
- Regular security updates and monitoring are necessary to protect application data.

**10. Limited Notification Facilities**

- Users may not receive instant notifications about booking confirmation, cancellation, or payment updates if notification services are not integrated.
- Additional email or SMS services may be required to provide automated alerts.

## 7.3 Future Enhancements

The Smart Parking Management System (City Park) provides a digital platform for managing parking slots, bookings, and payment-related activities. Although the current application offers essential parking management features, several additional technologies and functionalities can be implemented in the future to make the system more advanced, automated, secure, and efficient.

The following enhancements can be considered for future development:

### 1. IoT-Based Smart Parking Sensors

- IoT sensors can be installed in individual parking spaces to detect whether a vehicle is present.
- The sensors can automatically update parking slot availability.
- Users can receive more accurate and real-time parking information.
- This can reduce the need for manual slot status updates.

### 2. Automatic Number Plate Recognition (ANPR)

- Camera-based technology can be integrated to recognize vehicle registration numbers.
- The system can automatically record vehicle entry and exit details.
- Vehicle information can be matched with existing booking records.
- This can improve vehicle identification and parking security.

### 3. GPS-Based Parking Location

- GPS technology can help users identify nearby parking facilities.
- The application can display parking locations on an interactive map.
- Users can find parking spaces based on their current location.
- Navigation support can help users reach the selected parking facility.

### 4. Advanced Online Payment Integration

- Additional payment gateways can be integrated into the application.
- Users can make payments through UPI, debit cards, credit cards, and digital wallets.
- Automatic payment confirmation and digital receipts can be provided.
- Refund processing can be improved for cancelled bookings.

### 5. QR Code-Based Entry and Exit

- A unique QR code can be generated for every confirmed booking.
- Users can scan the QR code at the parking entrance.
- The system can verify booking details before allowing entry.
- QR code scanning can also be used during vehicle exit.

### 6. Automated Parking Barriers

- Automatic entry and exit barriers can be connected to the application.
- The barrier can open after successful booking verification.
- This can reduce manual intervention at parking entrances.
- It can improve vehicle movement and parking facility management.

### 7. SMS and Email Notifications

- Automatic booking confirmation messages can be sent to users.
- Users can receive payment confirmation and cancellation updates.
- Notifications can be sent before the booking time expires.
- Administrators can receive alerts about parking availability and system activities.

### 8. AI-Based Parking Prediction

- Artificial Intelligence can be used to analyze previous parking data.
- The system can predict parking demand during busy hours.
- Users can receive suggestions about suitable parking locations.
- Administrators can use predictions to improve parking space utilization.

### 9. Multiple Parking Location Management

- The application can be expanded to support multiple parking facilities.
- Users can compare parking availability at different locations.
- Administrators can manage several parking areas through a single dashboard.
- The system can support shopping malls, offices, hospitals, colleges, and public parking areas.

### 10. Advanced Admin Analytics and Reports

- Administrators can access detailed parking usage reports.
- Daily, weekly, and monthly booking statistics can be generated.
- Revenue reports can help monitor payment transactions.
- Graphs and charts can display parking slot utilization.
- Reports can help administrators make better management decisions.

# 8. REFERENCES / BIBLIOGRAPHY

## 8.1 References


- Python Software Foundation, Python Documentation: https://docs.python.org/3/.
- Oracle, MySQL Reference Manual: https://dev.mysql.com/doc/.
- Object Management Group, UML Specification: https://www.omg.org/spec/UML/.
- Somerville, Ian, Software Engineering, 10th edition, Pearson Education.

# 9. APPENDIX

## A) Application Startup Configuration

Application Startup Configuration describes the initial setup and configuration required to launch and access the Smart Parking Management System. It explains the steps involved in starting the application, accessing its interface, and preparing the system for users and administrators.

City Park is a web-based Smart Parking Management System developed using the Base44 platform. It can be accessed through a web browser using its deployed application URL.

Application URL: https://citypark.base44.app

- **System Startup Requirements**

Before starting the application, the following requirements should be available:

| Sr. No. | Requirement | Description |
|---|---|---|
| 1 | Operating System | Windows 10 / Windows 11 or equivalent. |
| 2 | Web Browser | Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari. |
| 3 | Internet Connection | Stable internet connectivity. |
| 4 | Application URL | Deployed CityPark application link. |
| 5 | User Account | Registered user credentials. |
| 6 | Admin Account | Authorized administrator credentials. |
| 7 | Hosting Platform | Base44 application hosting environment. |

### 6.1.2 Application Startup Procedure

The following steps are used to start and access City Park:

Step 1: Start the System

- Switch on the computer, laptop, or smartphone.
- Ensure that the device is connected to the internet.
- Open a supported web browser.

Step 2: Open the Application

- Enter the City Park application URL in the browser's address bar.
- Press Enter to open the application.
- Wait for the application homepage to load.

Step 3: Access the Login Page

- Open the login section of the application.
- Select the appropriate login option.
- Enter the registered email address and password.
- Click the Login button to continue.

Step 4: User Dashboard

After successful user authentication, the system redirects the user to the User Dashboard.

The user can perform the following operations:

- View available parking slots.
- Check parking slot status.
- Book an available parking space.
- Complete the payment process, if supported.
- View booking and payment information.

Step 5: Admin Dashboard

The administrator logs in using authorized admin credentials. After successful authentication, the system opens the Admin Dashboard.

The administrator can perform the following operations:

- View parking slot information.
- Add or update parking slots.
- Manage parking availability.
- Monitor user bookings.
- View payment records.
- Manage parking-related information.

## B) Main Application Modules

The Smart Parking Management System (City Park) is a web-based application developed to simplify and improve the management of parking facilities. The system provides separate interfaces for administrators and users to perform their respective operations.

The application is divided into different modules, where each module performs a specific function. These modules work together to provide an organized parking management process, from user registration and login to checking parking availability, booking parking slots, managing payments, and maintaining parking records.

The main purpose of modular development is to make the application easier to use, maintain, manage, and upgrade.

- **Main Modules of the Application**

The Smart Parking Management System consists of the following major modules:

1. Home Page Module
2. User Registration Module
3. Login and Authentication Module
4. User Dashboard Module
5. Parking Slot Management Module
6. Parking Slot Availability Module
7. Parking Booking Module
8. Payment Management Module
9. Admin Dashboard Module
10. Booking and Session Management Module
11. Logout Module

## C) How to Run the Project

The Smart Parking Management System, named City Park, is a web-based application developed using the Base44 platform. The application allows users to access parking facilities through a web browser without requiring a separate software installation.

The system provides separate access for users and administrators. Users can check parking slot availability, book parking spaces, and access payment-related features. Administrators can manage parking slots, monitor bookings, and manage parking-related information.

Follow the steps below to access and run the Smart Parking Management System.

Step 1: Start the Computer

- Switch on the computer or laptop.
- Make sure the operating system is working properly.
- Connect the device to an active internet connection.

Step 2: Open a Web Browser

- Open Google Chrome, Microsoft Edge or any supported web browser.
- Ensure that the browser is updated to the latest version.

Step 3: Open the Application

Enter the following application URL in the browser's address bar:

https://citypark.base44.app

- Press Enter.
- Wait for the application to load.
- The City Park home page will appear.

Step 4: User Registration

New users must create an account before accessing user services.

1. Open the registration page.
2. Enter the required details, such as name, email address and password.
3. Complete the registration process.
4. After successful registration, proceed to the login page.

*Existing registered users can directly proceed to login.*

Step 5: User Login

1. Open the user login page.
2. Enter the registered email address.
3. Enter the correct password.
4. Click the Login button.
5. After successful authentication, the User Dashboard will be displayed.

Step 6: Access the User Dashboard

After login, users can access the available parking-related features.

The user can perform the following operations:

- View available parking slots.
- Check parking slot status.
- Select a suitable parking slot.
- Create a parking booking.
- Access payment-related options.
- View booking information and records, where available.
- Log out after completing the activities.

Step 7: Admin Login

The administrator can access the administrative features through the separate admin login interface.

1. Open the Admin Login page.
2. Enter the authorized administrator credentials.
3. Click the Login button.
4. After successful authentication, the Admin Dashboard will appear.

Step 8: Access the Admin Dashboard

The administrator can perform the following operations:

- View the parking management dashboard.
- Add and manage parking slots.
- Update parking slot information.
- Change parking slot status.
- View and manage booking records.
- Monitor payment-related information.
- Manage parking activities.

Step 9: Logout

- Click the Logout option.
- The current session will be terminated.
- The application will redirect the user to the login page or home page.